ITM.690 인공지능 경영과 법 · 2026 가을 · 7강 · 실습 (2)

웹 서비스 배포: 검색엔진, RAG, 인증, 로깅

3강에서 만든 판결문 임베딩으로 유사 판례 검색엔진과 근거를 인용하는 RAG 챗봇을 만들고, Vercel로 공개 URL에 배포한 뒤 이메일 인증과 Supabase 로그를 붙인다. RAG의 한계를 실증 자료로 확인하고, 내 업무 중 AI로 자동화할 대상을 고르는 방법과 과제를 정리한다.

2026. 10. 15.

김병필 · KAIST 기술경영전문대학원

2026 에이전틱 AI 부트캠프 3주차 자료 기반

1 **Claude Code 시작하기**
git · gh · 저장소 · 권한

2 **검색엔진 구현**
키워드 vs 임베딩 · 데모 · 계획서

3 **RAG 적용**
근거 인용 챗봇 · 청킹 · Long Context

4 **RAG의 한계**
환각 · 검색 실패 · 보안 · 사고 사례

5 **Vercel 배포**
서버리스 · 환경변수

6 **인증과 권한**
해시 · 이메일 인증코드 · 고급 기법

7 **로깅: Supabase**
문답 기록 · 개인정보

8 **자동화 대상 고르기**
과업 분해 · 가치 × 위험

9 **정리와 과제**
10/29 수강생 발표

들어가며

오늘의 구성: 만들어 보고, 내 업무에 옮긴다

A

서비스 만들기 (§1~7)

Claude Code 준비 → 검색엔진 → RAG 챗봇 → RAG의 한계 → Vercel 배포 → 인증과 권한 → Supabase 로그. 부트캠프 3주차 실습을 강의 형식으로 옮기고, RAG의 한계와 인증 기법을 보충

B

자동화 대상 고르기 (§8)

내 업무를 과업 단위로 나누고, 가치와 위험으로 후보를 고른다. 스킬로 할지 웹앱으로 할지 정한다

C

과제 (§9)

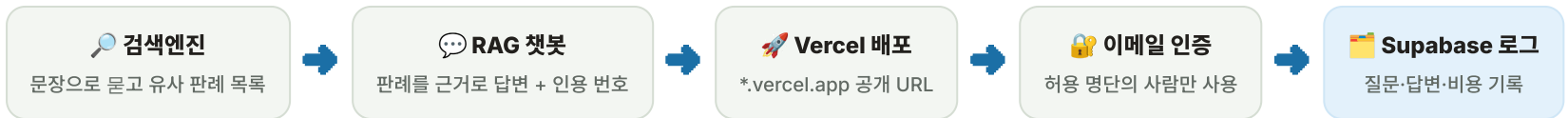
10/29(9강) 발표. 스킬 1건 + Vercel 웹앱 1건. "이것을 해도 되는가"의 애매한 점과 극복 방법을 함께 발표

3강과 달라지는 점: 3강의 결과물은 **정적 페이지**(GitHub Pages)였다. 오늘의 결과물은 **서버가 있는 서비스**다. 서버가 생기면 비밀(API 키) 보관, 접근 통제, 기록 보관이라는 문제가 함께 생긴다. 6강에서 다룬 개인정보 보호법이 이 지점에서 실무 문제가 된다.

원문 자료: 2026 에이전틱 AI 부트캠프 3주차: 검색엔진과 RAG

들어가며

오늘 만드는 것: 판결문 347건으로 공개 서비스까지



- **재료**: 3강에서 수집한 직장 내 괴롭힘 손해배상 판결 347건(요약 + 본문)과 임베딩 JSON 347개. 3강 과제의 특허·논문 임베딩이 있으면 그것을 써도 된다
- **도구**: Claude 데스크톱 앱의 Code(Claude Code). 서버를 내 PC에서 켜 두고, 코드를 깃헙에 올려야 하기 때문
- **계정**: GitHub(3강), OpenRouter 키(3강), Vercel·Supabase(GitHub 계정으로 가입), Gmail 앱 비밀번호

> **완성본 예시**: 부트캠프 실습의 실제 배포 결과가 no-bullying-gamma.vercel.app에 공개되어 있다. 오늘 슬라이드의 화면과 기록은 이 서비스를 만들면서 나온 실제 기록이다.

SECTION 1

Claude Code 시작하기

git·gh·파이썬을 설치하고, 작업 폴더를 Claude Code로 열어 지시 한 번으로 깃헙 저장소를 만든다. 자동 승인을 어디까지 허용할지도 정한다.

1 Claude Code 시작하기
2 검색엔진 구현
3 RAG 적용
4 RAG의 한계
5 Vercel 배포
6 인증과 권한
7 로깅: Supabase
8 자동화 대상 고르기
9 정리와 과제

CLAUDE CODE 시작하기

설치: git · gh · 파이썬

git

변경 이력 기록

파일이 언제 어떻게 바뀌었는지 기록하고 깃헙 저장소와 주고받는다. Claude Code가 코드를 저장·업로드할 때 내부에서 쓴다

gh

터미널에서 깃헙 조작

GitHub 공식 명령줄 도구. "저장소를 만들어 줘"라는 지시를 Claude Code가 실행할 때 이 도구를 쓴다

python

검색 서버 실행

파이썬 코드는 Claude Code가 쓴다. 다만 그 코드를 실행할 파이썬은 내 PC에 있어야 한다

```
# Windows (PowerShell)
winget install --id Git.Git -e
winget install --id GitHub.cli -e
winget install --id Python.Python.3.13 -e
# macOS (터미널)
git --version ; python3 --version ; brew install gh
```

```
# 설치 확인: 셋 다 버전 번호가 나오면 성공
git --version
gh --version
python --version
# 깃헙 로그인: 브라우저가 열리고 일회용 코드 입력
gh auth login --hostname github.com --git-protocol https --web
```

앱 설치는 AI에게 물어 가며 진행한다. 오류 메시지나 설치 화면을 Claude에 그대로 붙여넣고 "다음에 뭘 해야 해?"라고 묻는다. 예: Windows에서 명령을 찾지 못하면 터미널 창을 새로 열기, brew가 없으면 Claude Code에 "gh를 설치해줘"라고 요청

gh 로그인: 명령 한 줄이면 브라우저가 열린다

질문에 Yes로 답하고, 화면에 뜬 일회용 코드를 복사한 뒤 Enter를 누르면 브라우저가 열린다. 브라우저에 코드를 붙여넣고 승인하면 끝난다.

```
PS C:\claude\no-bullying> gh auth login --hostname github.com --git-protocol https --web
? Authenticate Git with your GitHub credentials? Yes
! First copy your one-time code: 1BC6-A269  ← 이 코드를 복사해 브라우저에 붙여넣는다
Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete.
- gh config set -h github.com git_protocol https
✓ Configured git protocol
✓ Logged in as kaist-itm-aibootcamp
```

- 어느 폴더에서 실행해도 된다. 로그인은 폴더가 아니라 계정 단위다
- 잘 되었는지는 `gh auth status`로 언제든 확인한다. “Logged in as” 뒤의 계정명이 내 계정인지 본다
- 브라우저 기반 일회용 코드 방식(device flow)은 비밀번호를 터미널에 입력하지 않는 인증 방법이다

CLAUDE CODE 시작하기

작업 폴더를 열고, 지시 한 번으로 깃헙 저장소 만들기

- 프로젝트 폴더를 만든다. 예: `C:\claude\no-bullying` (macOS `~/claude/no-bullying`). 3강 산출물(판결문 `rulings/`, 임베딩 `embeddings_json/`, 요약 `rulings_summary.csv`)과 OpenRouter 키 파일을 이 안으로 옮긴다
- 데스크톱 앱에서 **Code**를 고르고 이 폴더를 작업 폴더로 연다. Cowork와 달리 AI가 이 폴더 안에서 프로그램을 직접 실행한다

💬 프롬프트

현재 폴더 내용으로 깃헙 레포를 만들어서 작업하려고 해.
깃헙에 no-bullying이라는 레포를 만들어줘.

복사

- Claude Code가 git 초기화, 제외 파일 목록(`.gitignore`) 작성, gh로 저장소 생성과 업로드까지 진행한다. 공개 범위는 **비공개(private)**로 시작한다

올리기 전에 직접 확인: github.com에서 저장소를 열어 키 파일이 없는지 본다. 이미 올라갔다면 파일을 지우는 것으로는 부족하다. 커밋 이력에 남기 때문이다. 해당 키를 **폐기하고 새로 발급**한다. 공개 저장소의 키를 자동으로 수집하는 프로그램이 상시 동작한다.

권한: 자동 승인을 어디까지 허용할 것인가

방식	동작	어울리는 경우
매번 확인	명령 실행·파일 변경마다 내용을 읽고 승인	처음 쓰는 도구, 낯선 폴더, 민감한 자료
선택적 허용	같은 종류의 작업은 계속 허용	반복 작업이 많은 실습 중반부터
자동 승인	대부분의 작업을 확인 없이 진행. 대량 삭제·외부 전송 같은 위험 작업은 따로 확인을 요청	신뢰하는 폴더의 학습·개인 프로젝트

- 오늘의 자료는 법원이 비실명화해 공개한 판결문이고, 잘못되어도 잃는 것은 폴더 하나다. 자동 승인을 켜도 된다
- 회사 문서, 고객 정보가 든 폴더라면 다르다

SECTION 2

검색엔진 구현: 단어가 아니라 의미로 찾기

키워드 검색의 한계를 확인하고, 임베딩 유사도 검색엔진의 구조(색인과 검색)를 이해한 뒤,
계획서부터 대화로 Claude Code에게 구현을 맡긴다.

1 Claude Code 시작하기
2 검색엔진 구현
3 RAG 적용
4 RAG의 한계
5 Vercel 배포
6 인증과 권한
7 로깅: Supabase
8 자동화 대상 고르기
9 정리와 과제

검색엔진 구현

두 가지 검색 방식: 키워드 매칭 vs 임베딩 유사도

①

키워드 매칭

- 질의어가 문서에 글자 그대로 있는지 확인한다
- 구현이 쉽고 서버 없이 브라우저만으로 동작한다
- 전통적 검색의 기본 방식

②

임베딩 유사도

- 질의와 문서를 같은 좌표계의 벡터로 바꾸어 거리를 잰다
- 단어가 달라도 의미가 같으면 찾는다
- 질의를 실시간으로 임베딩해야 하므로 **API 호출이 필요**하다

판결문 요약 347건에 키워드 검색을 해 보면 세 가지 실패가 나타난다

1

동의어를 모른다

"폭언" 90건, "욕설" 44건. 같은 현상인데 판결문이 어느 단어를 썼는지에 따라 결과가 갈린다

2

표현이 다르면 놓친다

"예견가능성"은 5건. "회사가 알 수 있었다"라고 쓴 판결은 뜻이 같아도 찾지 못한다

3

의도를 모른다

"괴롭힘을 인정하지 않은 사건"을 검색하면 "괴롭힘", "인정"이 많이 나오는 인정 사건이 위로 올라온다

검색엔진 구현 · 직접 써 보기

데모 ① 키워드 검색기: 세 가지 실패를 직접 확인한다

🔎 키워드 검색기 (판결 요약 347건 대상 · 브라우저에서만 동작)

폭언 | 검색

욕설 | 폭언 | 예견가능성 | 회사가 알 수 있었다 | 괴롭힘을 인정하지 않은 사건

질의어 **[폭언]** · 한 번이라도 등장한 판결 **90건** / 347건 · 등장 횟수 순 상위 8건 표시

청주지방법원 2022가단57865 · 2023 원고일부승 — 일치 3회

산업기능요원으로 근무하던 원고가 "귓방망이", "개돼지" 등 폭언을 한 생산관리부장과 회사를 상대로 치료비·위자료 약 4,146만 원을 청구했다. 법원은 폭언을 직장 내 괴롭힘에 의한 불법행위로 인정했으나 원고가 폭언을 유도한 정황 등을 참작해 위자료 50만 원만 인용했다.

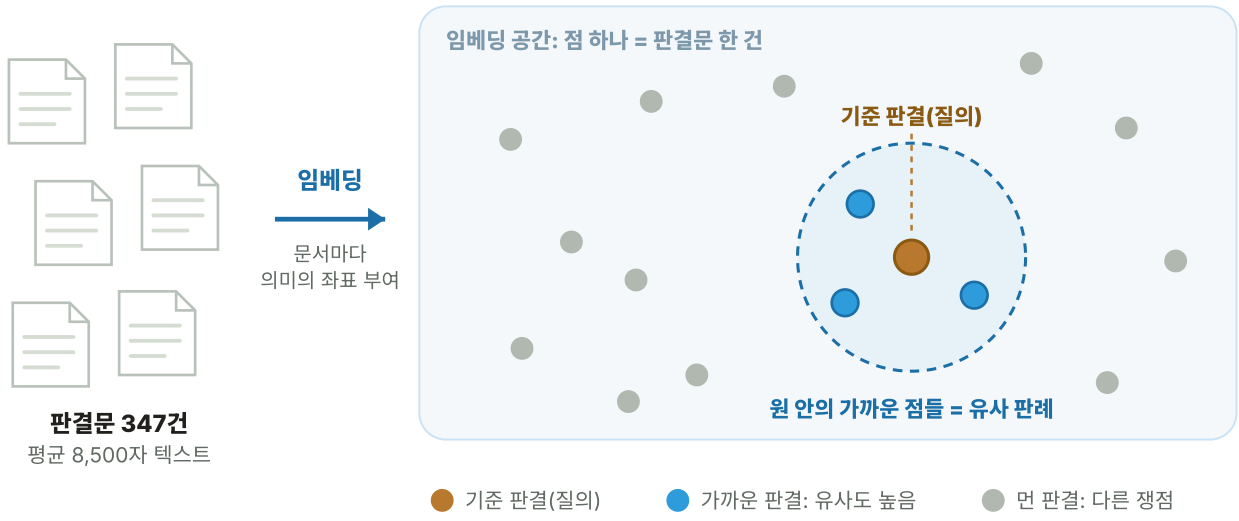
서울동부지방법원 2023가합106205 · 2025 원고일부승 — 일치 2회

치과 치위생사가 부장·실장의 반복적인 폭언·욕설 등 직장 내 괴롭힘과 대표원장의 사용자책임을 주장하며 휴업손해와 위자료 등 약 1,620만 원의 손해배상을 청구하였다. 법원은 부장·실장의 폭언을 직장 내 괴롭힘에 의한 불법행위로 인정하여 피고들이 공동하여 위자료 300만 원을 지급하라고 명하고 휴업손해 청구는 기각하였다(원고일부승).

질의어가 요약문에 글자 그대로 몇 번 나오는지 세어 많이 나온 순으로 보여준다. “폭언”과 “욕설”, “예견가능성”과 “회사가 알 수 있었다”를 차례로 눌러 결과를 비교해 본다.

검색엔진 구현

임베딩 유사도: “비슷한 판결 찾기”를 “가까운 점 찾기”로



- 판결문마다 4,096차원 벡터가 있다(3강에서 추출). 두 벡터의 **코사인 유사도**가 “의미가 얼마나 비슷한가”를 숫자로 나타낸다
- 질의 문장도 같은 방식으로 벡터로 만들면 문장으로 검색할 수 있다. 실제 점검에서 “업무상 적정범위를 넘지 않았다고 보아 괴롭힘을 부정한 사례”라고 묻자 1·2위가 그 논리로 기각된 원고패 판결이었다. 단어가 아니라 **판시 논리**가 검색된 것이다

검색엔진 구현 · 직접 써 보기

데모 ② 유사 판례 찾기: 실제 코사인 유사도 (347건 전건 계산)

🧭 기준 판결을 고르면 본문 임베딩이 가장 가까운 판결 5건을 보여준다

서울중앙지방법원 2021가단5297485 · 2023 · 원고일부승 ⌄ 🎲 무작위 판결

기준 판결: **서울중앙지방법원 2021가단5297485** 원고일부승 인사분쟁 복합형

채용비리 공익신고를 한 사회복지사가 센터장·팀장 등이 보복성 고충신고와 부당징계(감봉·보직변경)를 주도하는 등 직장 내 괴롭힘을 하였다며 운영법인들과 개인들을 상대로 위자료를 청구하였다. 법원은 센터장 주도의 보복성 징계 과정을 불법행위로 인정하여 법인과 센터장에게 1,500만 원(팀장…

본문 임베딩 코사인 유사도 상위 5건:

서울서부지방법원 2022가단289504 · 2024 원고패 인사분쟁 복합형 유사도 0.859

사회복지사인 원고가 팀장의 폭언·고압적 질책 등 직장 내 괴롭힘과 공익신고 후 불이익조치를 이유로 팀장과 재단을 상대로 위자료 3,200만 원을 청구하였다. 법원은 노동청의 괴롭힘 인정에도 불구하고 팀장의 언행이 업무상 적정범위를 넘었다고 보기 어렵고 사직 처리도 불이익조치가 아니라며 청구를 모두 기각하…

서울중앙지방법원 2022가합525998 · 2024 원고일부승 인사분쟁 복합형 유사도 0.840

위안부 피해자 지원 사회복지법인의 비리를 공익신고한 직원 7명이 시설장 등 운영진과 법인 등을 상대로 보복성 고소, 업무배제, 감시 등 불이익조치와 직장 내 괴롭힘을 이유로 각 1억 원의 손해배상을 청구한 사건이다. 법원은

이 데모는 왜 서버 없이 동작하는가: 347건끼리의 유사도는 미리 계산해 페이지에 넣어 둘 수 있기 때문이다. 문제는 사용자가 **새 질의문**을 입력하는 순간 시작된다. 그 문장의 벡터는 미리 계산해 둘 수 없다. 다음 장의 주제다.

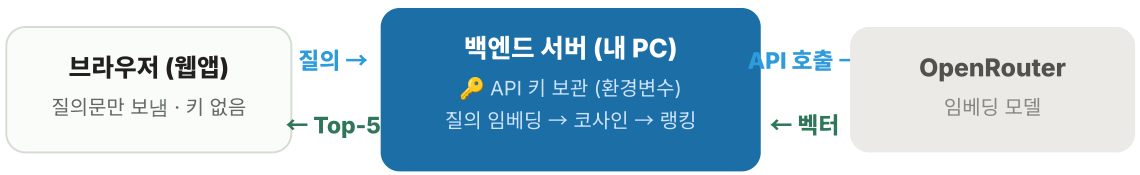
검색엔진 구현

새 질의를 받으려면 백엔드가 필요하다

- 347건끼리의 유사도는 미리 계산해 페이지에 넣어 둘 수 있다. 그러나 사용자가 새로 입력한 문장의 벡터는 미리 계산할 수 없다. 검색할 때마다 임베딩 API를 불러야 한다

문제 1. 호출마다 비용
질의 하나는 1원 미만이지만, 공개된 웹앱이라면 누가 몇 번 검색할지 통제할 수 없다. 비용은 내 크레딧에서 나간다.

문제 2. 키를 숨길 곳이 없다
브라우저 자바스크립트에 API 키를 넣으면 누구나 개발자도구(F12)로 키를 복사해 내 크레딧으로 호출할 수 있다. 정적 웹페이지에는 비밀을 둘 곳이 없다.



사용자 입력을 실시간으로 처리하고 비밀(키)을 지키려면 키를 가진 중간 프로그램, 백엔드가 필요하다

검색엔진 구현

임베딩 유사도 검색은 이미 널리 쓰이는 산업 표준이다

KIPRIS 유사 특허 검색

발명의 설명을 넣으면 표현이 달라도
기술적으로 유사한 선행 특허를 찾는다

유사 판례·조항 검색

리걸테크의 “이 판결과 유사한 판례”,
계약서의 유사 조항 검색

관련 논문 탐색

초록 임베딩으로 관련 논문 추천, 심사위원 매칭

고객센터 FAQ 매칭

표현이 달라도 같은 뜻의 과거 문의와 답변을 찾아 상담원에게 제시

표절·중복 탐지

지나치게 비슷한 문서 찾기, 같은 사건을 다룬 기사 묶기

매칭 서비스

이력서와 채용공고처럼 서로 다른 두 종류의 문서를 같은 좌표계에 놓고 비교

사내 문서 검색

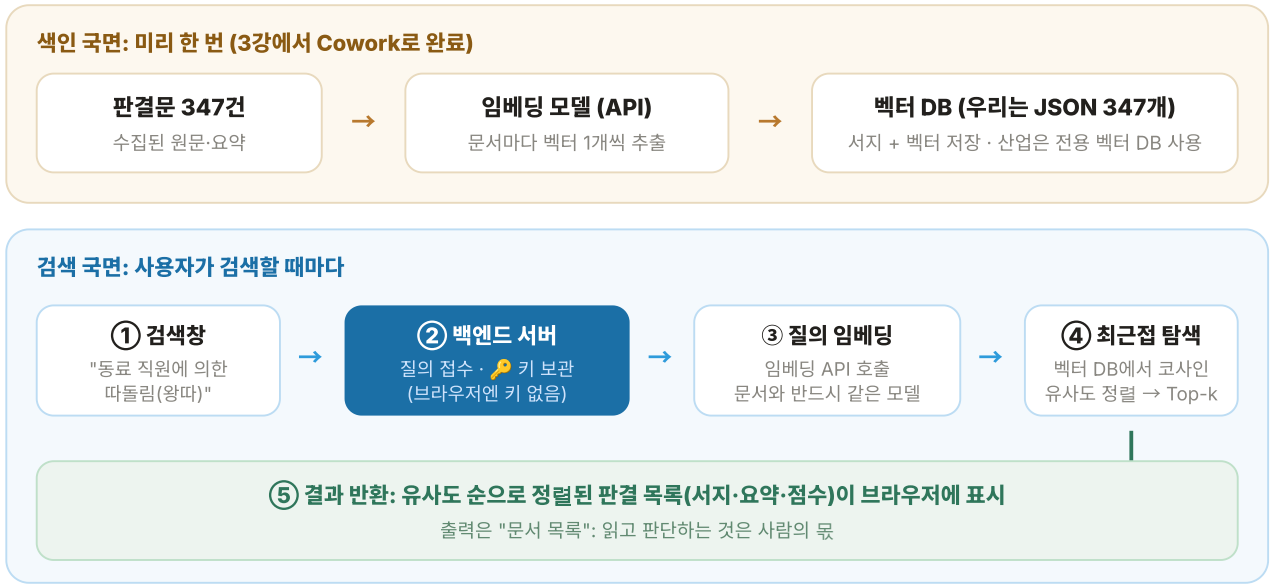
키워드 매칭에 의미 검색을 결합한
하이브리드 검색이 일반화되었다

챗봇의 검색 단계

기업 챗봇이 답하기 전에 사내 문서를
찾아오는 단계. §3의 RAG

검색엔진 구현

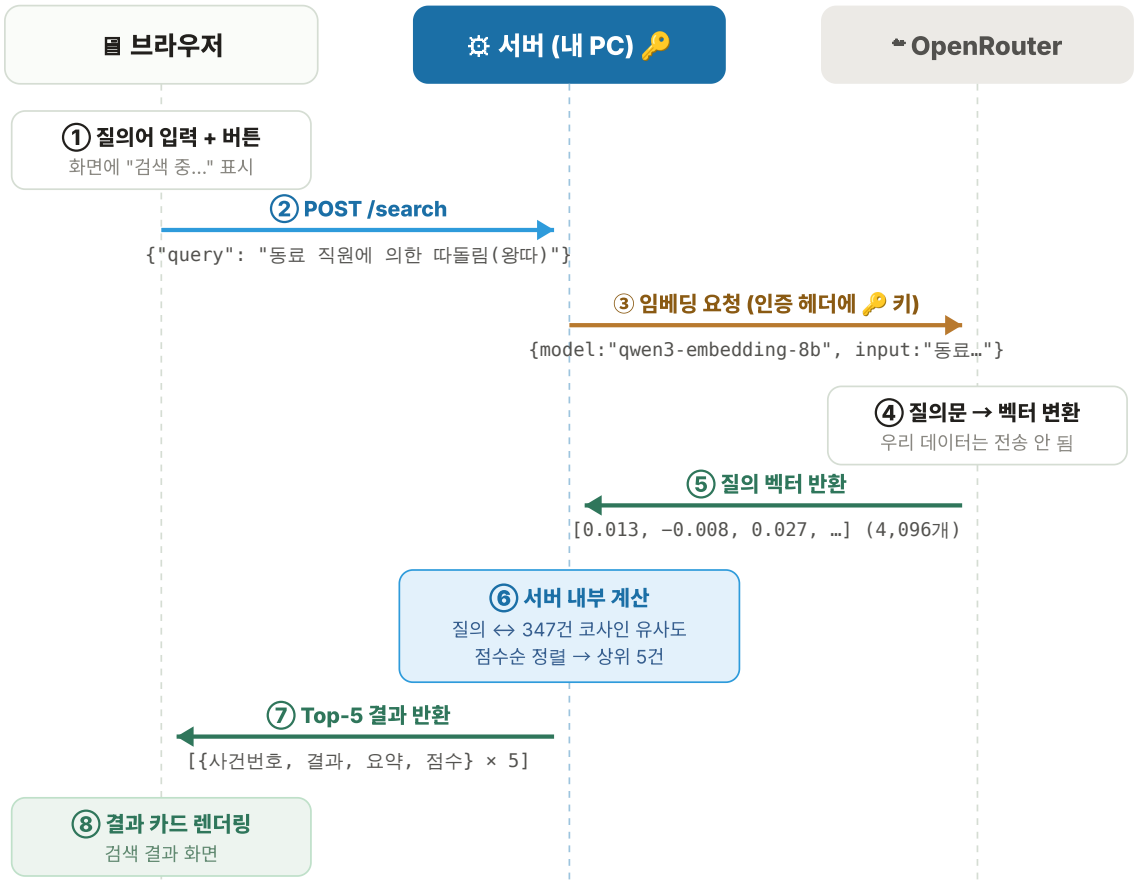
전체 구조: 색인(미리 한 번)과 검색(질의마다)



- **질의 임베딩 모델은 색인 때와 같은 모델**이어야 한다. 모델이 다르면 좌표계가 달라 유사도가 의미를 잃는다
- 347건이면 전수 비교로 충분하다. 수백만 건이면 전용 벡터 DB의 근사 최근접 탐색이 이 단계를 대신한다. 구조는 같다
- 출력은 **문서 목록**이다. 읽고 판단하는 것은 사람이 한다

검색엔진 구현

질의 하나의 데이터 플로우: “동료 직원에 의한 따돌림(왕따)” 검색의 1초



- 세로 방향이 시간 순서, 상자 안은 각 주체가 하는 일, 화살표 위는 실제로 오가는 데이터다
- 🔑 키는 ③에서 서버가 붙인다. 브라우저에는 끝까지 등장하지 않는다
- OpenRouter에는 **질의문 한 줄**만 간다. 판결문 347건은 내 PC를 떠나지 않는다
- 전체 소요는 1초 안팎이고, 대부분이 ③~⑤의 왕복이다
- 업무에 적용할 때 이 그림을 먼저 그려 본다. 어떤 데이터가 어느 회사 서버로 가는지가 “해도 되는가” 판단의 출발점이다

검색엔진 구현

“벡터 DB”의 실물: 문서 id와 벡터의 대응표

우리 것: 폴더 하나에 JSON 347개 (파일당 평균 181KB)

```
{
  "id": "수원지방법원_2020가단8667",
  "법원": "수원지방법원", "사건번호": "2020가단8667",
  "선고일": "2021. 4. 21.", "소송결과": "원고일부승",
  "embedding_model": "qwen/qwen3-embedding-8b",
  "embedding_dim": 4096,
  "body_embedding": [-0.01119, -0.00574, 0.03813, …],  // 숫자 4,096개
  "summary": "원고가 노동조합 조합장인 피고를 상대로…",
  "summary_embedding": [-0.03728, -0.00944, …]
}
```

일반적 서비스: 전용 벡터 DB (예: PostgreSQL + pgvector)

```
-- rulings 테이블: 문서 1건 = 1행, 벡터 칸이 하나 더 있는 표
 id | 법원          | 사건번호       | embedding vector(4096)
  1 | 수원지방법원 | 2020가단8667  | [-0.0112, -0.0057, …]
  2 | 대구지방법원 | 2024가단138550| [ 0.0093, -0.0141, …]

-- 검색: 가까운 순으로 정렬해 5건
SELECT 법원, 사건번호 FROM rulings
ORDER BY embedding <=> [질의 벡터]  -- 벡터 거리 연산자
LIMIT 5;
```

Pinecone, Chroma, Milvus, pgvector 등. 전용 DB의 가치는 수십만 건 이상에서 전수 비교 없이 빠르게 정렬하는 색인 기술에 있다.

검색엔진 구현

구현 방법: 사양서를 쓰지 말고 계획서부터 대화로



💬 프롬프트

복사

나는 개발자가 아니야. 바로 코드를 짜지 말고, 계획서부터 만들어줘.
내가 원하는 것:
- 이 폴더의 embeddings_json/ 안에 판결문 347건의 임베딩 JSON이 있어
- 검색창에 문장으로 질의하면, 의미가 비슷한 판결을 순서대로
  보여주는 검색 서비스를 내 컴퓨터에서 돌리고 싶어

코드를 읽지 못해도 계획서는 읽고 고칠 수 있다. 비개발자가 개발을 통제하는 지점이 계획서다. 사람 개발자에게 외주를 줄 때와 같은 방식이다.

검색엔진 구현

AI의 질문에 답하고, 요구를 계획서에 명문화한다

계획서 v1 (요약): 작은 웹서버 + 검색 화면. 데이터 통합 → 검색 엔진(화면 없이 먼저 검증) → 웹 화면 → 실행 방법. 질문 3가지: (a) 본문 임베딩만 쓸까, 요약 임베딩과 조합할까 (b) 법원·연도 필터를 넣을까 (c) 질의 임베딩 모델을 지정해 줄 수 있나

💬 프롬프트

복사

```
(a) 본문 요약 임베딩 조합
(b) 검색창만 있는 단순 버전
(c) qwen/qwen3-embedding-8b
```

확정된 계획서(PLAN.md)의 구성

만들 것	내 컴퓨터에서만 도는 판결문 의미 검색 웹페이지. 외부로 나가는 것은 “검색 문장을 임베딩해 달라”는 요청 하나뿐
확정 사항	본문+요약 임베딩 중 높은 쪽 · 필터 없음 · 같은 임베딩 모델 · DB 설치 없이 메모리 계산
확인 지점	① 347건 통합, 누락 0건 보고 ② 시험 검색 결과 승인 ③ 직접 검색해 보고 완료 판단
비용	검색 1회당 임베딩 호출 1번, 1원 미만
하지 않는 것	필터, 인터넷 공개, 로그인, 키워드 검색, 새 판결 자동 임베딩

확인 지점이 문서에 들어 있어 구현 내내 통제권이 사용자에게 남는다. “하지 않는 것”을 적어 두면 범위가 늘어나지 않는다.

검색엔진 구현

확정된 계획서(PLAN.md): 실제 실습에서 나온 그대로

판결문 유사 검색 서비스: 제작 계획서 작성일 2026-08-17 · 상태: 4단계 모두 완료 (사용 방법은 README.md 참고)

1. 무엇을 만드는가

- 내 컴퓨터에서만 돌아가는 판결문 의미 검색 웹페이지. 브라우저에서 http://localhost:8000을 열면 검색창이 하나 나온다
- 입력한 문장을 OpenRouter API로 임베딩한 뒤, 판결문 347건의 임베딩과 유사도를 계산해 가까운 순으로 보여준다. 결과마다 법원·사건번호·선고일·소송결과·사건 개요·유사도, 클릭하면 전문
- 외부로 나가는 것은 "검색 문장을 임베딩해 달라"는 요청 하나뿐이다. 판결문 원문이 통째로 외부에 전송되는 일은 없다

2. 확정된 사항

검색 비교 기준	본문 임베딩 + 요약 임베딩 중 높은 쪽으로 순위
화면 구성	검색창만 있는 단순 버전 (필터 없음)
임베딩 모델	기존 데이터와 같은 qwen/qwen3-embedding-8b. 같은 모델이어야 비교가 의미 있음
기술 방식	Python 웹서버 하나. DB 설치 없이 347건을 메모리에서 계산

3. 만드는 순서와 확인 지점

- **1단계 데이터 통합**: JSON 347개를 검색용 파일 하나로. ✅ 확인 지점 1: "347건 통합 완료, 누락 0건" 보고
- **2단계 검색 엔진 (화면 없이 먼저 검증)**: 테스트 문장 2~3개로 결과가 상식적인지 확인. ✅ 확인 지점 2: 결과를 보고 진행 여부 판단, 직접 시험할 문장이 있으면 이때 알림
- **3단계 웹 화면**: 검색창 + 결과 목록 + 전문 페이지
- **4단계 실행 방법 정리**: 명령 한 줄로 서버 켜기. ✅ 확인 지점 3(최종): 직접 검색해 보고 완료 판단

4. 준비물 점검 · 5. 비용과 시간

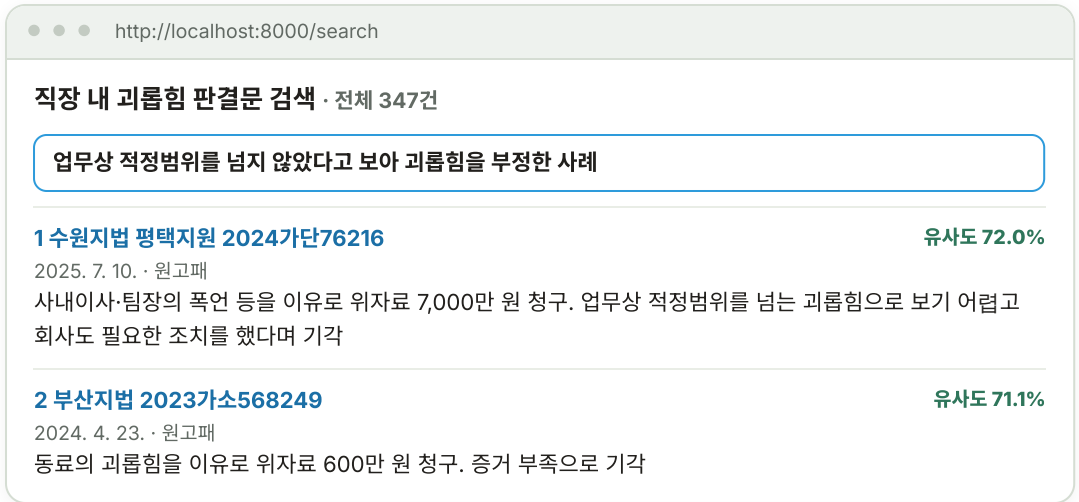
- 임베딩 JSON 347건(건당 본문·요약 각 4,096차원), 판결문 원문 347건, 요약 CSV, 키 파일, 파이썬: 모두 확인
- 검색 1회당 임베딩 호출 1번, 원화 몇 원 미만. 실작업은 한 세션 안에 끝나는 규모

6. 이번 버전에서 하지 않는 것

- 법원·연도·소송결과 필터, 인터넷 공개·배포, 로그인 기능, 키워드 검색, 새 판결문 추가 시 임베딩 자동 생성

검색엔진 구현

구현 결과, 그리고 써 보면서 기능을 덧붙인다



- 판결문은 스스로를 "부정한 사례"라고 부르지 않는다. 키워드 검색으로는 성립하지 않는 질의에 원고패 판결이 상위로 나왔다
- 시험 질의 "동료 직원들에 의한 따돌림(왕따)"에서 "왕따"라는 구어와 "집단 따돌림"이라는 판결문 표현이 달라도 찾아내는지 확인한다

💬 프롬프트

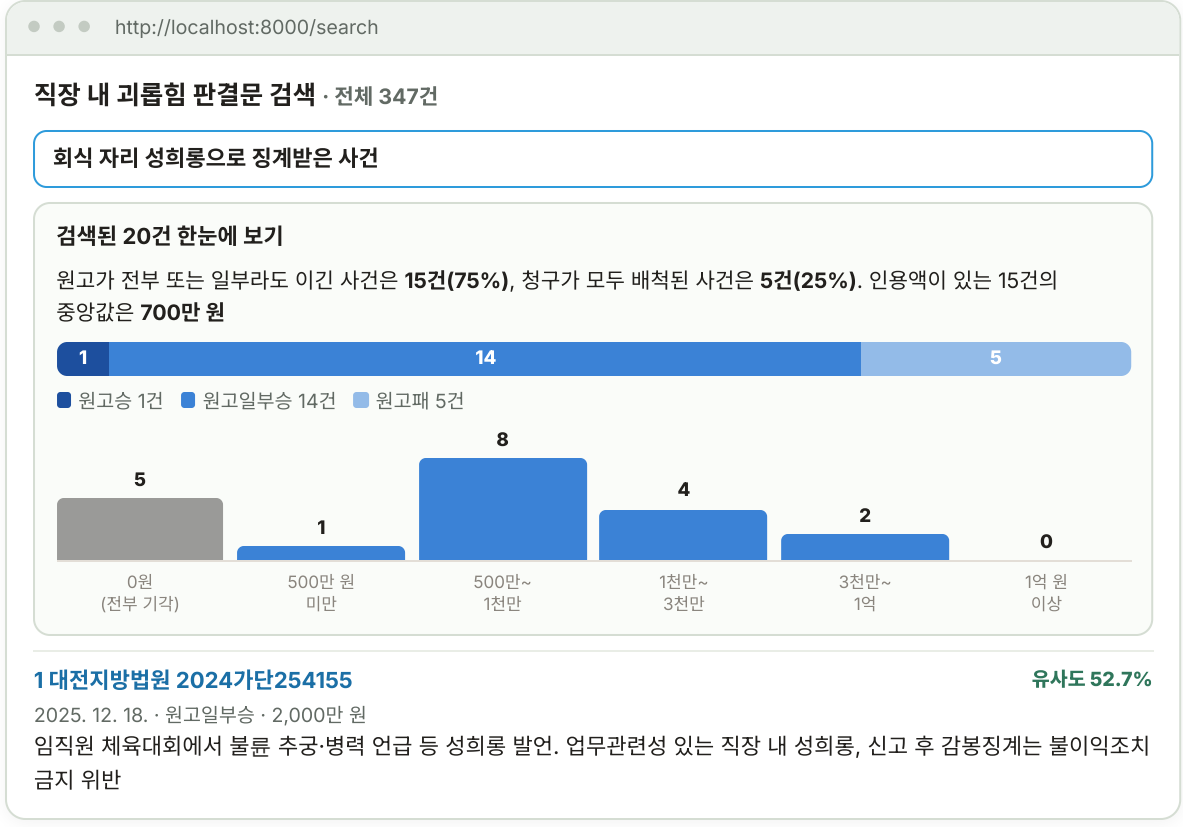
다음 2가지 기능을 추가한다.
(1) 판결문을 클릭했을 때 새창에 뜨는 것이 아니라 오른쪽 패널에 표시
(2) 검색된 가까운 20건에 대해서 원고 승패 비율과 인용된 금액 분포 히스토그램을 상단에 표시 - 곧바로 전체 현황을 알 수 있도록 설명

복사

"회식 자리 성희롱으로 징계받은 사건" 상위 20건: 원고가 일부라도 이긴 사건 75%, 인용액 중앙값 700만 원. 검색기가 분석 도구가 되었다. 완성은 "써 보고, 아쉬운 점을 말하고, 고치게 하는 것"의 반복이다.

검색엔진 구현

두 번째 결과물: 검색기가 분석 도구가 된다



- 앞 장의 지시 (1) 전문을 오른쪽 패널에 표시 (2) 상위 20건의 승패 비율과 인용액 분포를 상단에 요약. 이 두 줄로 나온 화면이다
- 목록을 읽기 전에 "이 유형의 사건에서 원고가 이긴 비율 75%, 인용액 중앙값 700만 원"이라는 전체 현황이 먼저 보인다
- 화면에는 **"인용액은 주문에서 지급을 명한 금액을 합산한 값이며, 원고가 여럿이고 '각 ○○원' 형태인 경우 등에는 실제와 차이가 있을 수 있다"**는 주의 문구가 붙어 있다. 자동 추출한 숫자의 한계를 화면에 밝히는 것도 설계의 일부다
- 비교 실험: 데모 ①의 키워드 검색기와 같은 질의("회사가 알 수 있었다" 등)를 던져 보면, 키워드가 놓치던 사건을 임베딩 검색이 찾는 것을 확인할 수 있다

검색엔진 구현

문서화: 코드만 남기지 말고 계획·결정·사용법을 남긴다

파일	담는 것	실무 표준으로 치면
PLAN.md	합의한 계획서의 최종본. 무엇을, 어떤 구조로, 어떤 순서로	설계 문서(design doc)
DECISIONS.md	날짜 + 결정 + 이유 + 검토했다가 버린 대안. 지우지 않고 아래로만 추가	결정 기록(ADR)
README.md	처음 보는 사람이 실행·배포까지: 필요한 것, 실행 명령, 환경변수 목록(이름과 용도만), 자주 나는 문제	README + 운영 문서
CLAUDE.md	Claude Code가 폴더에서 자동으로 읽는 프로젝트 메모. 폴더 구조, 규칙, 자주 쓰는 명령	팀 온보딩 문서

💬 프롬프트

오늘 작업을 문서로 남기자. 프로젝트 폴더에 PLAN.md,
DECISIONS.md, README.md, CLAUDE.md 네 파일을 만들어줘.
전부 코드에서 확인되는 사실만 쓰고, 확실하지 않은 것은
[확인 필요]로 표시해줘.

복사

💬 프롬프트

CLAUDE.md에 다음 규칙을 추가해줘:
"코드를 변경하면 관련된 문서(README.md, PLAN.md)를 같은
커밋에서 갱신한다. 방향을 정하는 결정을 내렸으면
DECISIONS.md에 항목을 추가한다."

복사

문서화가 실패하는 시점은 처음 만들 때가 아니라 고치지 않을 때다. 갱신 규칙을 CLAUDE.md에 두면 이후에는 AI가 코드와 문서를 함께 고친다. 이 기록은 §5에서 다시 쓰인다.

SECTION 3

RAG 적용: 검색한 문서를 근거로 답하게 하기

검색엔진이 문서 목록을 돌려준다면, RAG는 검색된 문서를 AI의 컨텍스트에 넣고 그 문서만 근거로 답변을 쓰게 한다. 근거 번호를 인용하고, 근거가 없으면 답하지 않는 판례 챗봇을 만든다.

1 Claude Code 시작하기
2 검색엔진 구현
3 RAG 적용
4 RAG의 한계
5 Vercel 배포
6 인증과 권한
7 로깅: Supabase
8 자동화 대상 고르기
9 정리와 과제

RAG 적용

RAG(Retrieval-Augmented Generation)란 무엇인가

- LLM에게 그냥 물으면 학습 데이터에 기초해 답한다. 이 방식에는 세 가지 한계가 있다: **학습 이후의 일을 모르고**, **내 데이터(사내 문서, 수집한 판결문)를 모르고**, 모르는 내용도 **그럴듯하게 지어낸다(환각)**
- RAG: 질문이 들어오면 관련 문서를 **검색**(Retrieval)해 질문과 함께 컨텍스트에 **추가**(Augmented)하고, 그 문서를 근거로 답을 **생성**(Generation)하게 한다

최신·내부 데이터

모델이 학습한 적 없는 문서(어제 수집한 판결문, 사내 규정)로 답할 수 있다

출처 표시

어느 문서를 근거로 답했는지 인용을 요구할 수 있어 검증이 가능해진다

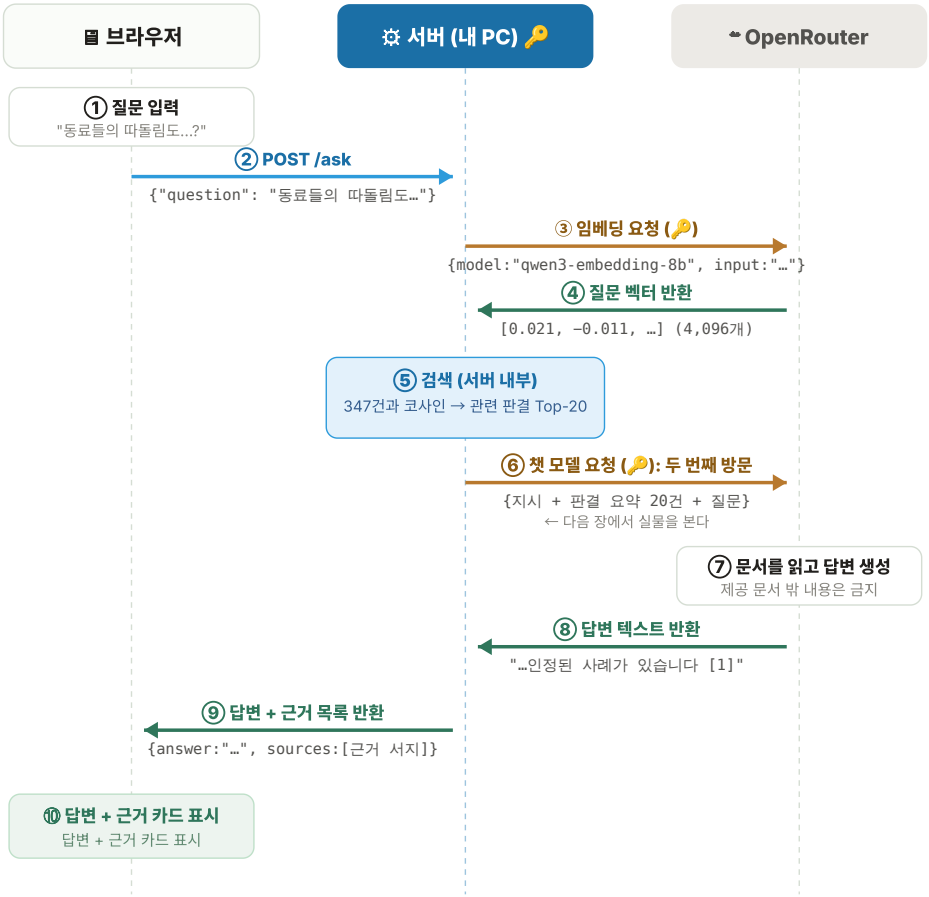
환각 억제

"제공된 문서로 답할 수 없으면 모른다고 하라"는 지시가 가능해진다. 다만 환각이 사라지지는 않는다(§4)

검색엔진과 RAG는 목적이 다르다. 검색엔진의 사용자는 문서 목록을 직접 읽고 판단하려는 사람이고, RAG의 사용자는 종합된 답을 원하는 사람이다. RAG는 검색엔진을 구성요소로 쓴다. 연구·실무에서는 대개 둘 다 필요하다.

RAG 적용

데이터 플로우: OpenRouter를 두 번(임베딩, 챗) 호출한다



- ③~⑤는 §2의 검색엔진과 같다. 질문과 가까운 판결 20건을 고른다
- ⑥에서 **지시 + 판례 요약 20건 + 질문**을 하나의 요청으로 챗 모델에 보낸다
- 키는 서버만 가진다. 판결문 347건 원본은 내 PC를 떠나지 않고, 검색된 20건의 요약만 전송된다
- 소요 시간은 십수 초로 늘어난다. 실측: 임베딩 1초 + 답변 생성 12초
- **데이터가 어디로 가는가**를 그림으로 그릴 수 있어야 6강의 처리위탁·국외 이전 문제를 판단할 수 있다

RAG 적용

왜 Chat·Cowork가 아니라 유료 API인가

Chat · Cowork: 사람이 쓰는 도구

- 사람이 로그인해 화면에서 직접 실행하는 개인용 서비스
- 내 연구·업무를 내가 할 때는 이것으로 충분하다
- 그러나 내 프로그램(서버)이 대신 호출할 수 없고, 계정 1인분이다. 이용자 100명을 받을 수 없다

외부 API: 프로그램이 부르는 인터페이스

- 서버가 자동으로, 동시에, 사람 없이 호출한다
- 쓴 만큼 과금. 이용자가 늘면 비용도 비례해 늘어나는 구조
- 챗봇·웹앱 같은 서비스는 일반적으로 이 구조를 쓴다

혼자 쓰는 분석이면 Chat·Cowork, 다른 사람이 쓰는 서비스를 만드는 순간 API

이 구분은 §8에서 업무 자동화의 형태(스킬 vs 웹앱)를 고르는 기준이 된다. 법적으로도 다르다: 개인 구독 서비스의 약관과 API 이용 약관은 데이터 학습·보존 조건이 다를 수 있다.

RAG 적용

“증강”의 실물: 서버가 보내는 요청 하나

```
POST https://openrouter.ai/api/v1/chat/completions
{ "model": "deepseek/deepseek-v4-flash", "temperature": 0.2, "max_tokens": 3000, "stream": true,
  "messages": [
    { "role": "system", "content": "아래 <판례>에 적힌 내용만 근거로 삼는다. 근거가 된 판례의 번호를 [1], [7]처럼 붙인다.
       <판례>만으로 답할 수 없으면 '제가 가진 347건의 판결문으로는 답변드릴 수 없습니다'라고 답한다. …" },
    { "role": "user", "content":
       "<판례>
        [1] 수원지방법원 안산지원 2022가단87835 · 원고일부승 · 333,751,696원
            팀장의 지속적인 공개 질책·모욕 등 직장 내 괴롭힘으로 우울증을 겪다 자살한 근로자의 유족이 …
        [2] … [20] (같은 형식, 약 4,900자)

        질문: 직장 내 괴롭힘으로 우울증이 생겼을 때 회사에도 책임을 물을 수 있나요?" } ] }
```

- **지시(system)**는 매번 같고, **근거 자료(user)**는 질문마다 새로 담긴다. **질문**은 40자, 근거는 5,252자. 전송량의 대부분이 근거 자료다
- 모델은 판결문 347건을 갖고 있지 않다. 매번 고른 20건만 보고 답한다. 검색이 문서를 잘못 고르면 답도 틀린다. **RAG 품질의 절반은 검색 품질**이고, 답이 이상하면 생성보다 검색 결과부터 확인한다

RAG 적용

실습: 계획 문답, 그리고 모르는 결정은 근거를 요구하며 위임한다

💬 프롬프트

복사

나는 개발자가 아니야. 바로 코드를 짜지 말고 계획서부터 만들어줘.
- 지금 만든 판례 검색 서비스에 챗봇 기능을 추가
- 질문을 하면, 판결문 347건에서 관련 판례를 찾아 그것을 근거로 답해주는 방식(RAG)
- 답변에는 어떤 판결을 근거로 했는지 표시
- 근거가 없으면 아는 척하지 말고 모른다고 답변

AI의 질문	실제 답
(a) 무엇을 근거로 읽게 할까	요약만 20건 (직접 선택)
(b) 답변 모델	"가성비 좋은 것으로 추천하고 이유를 계획서에 적어줘" (위임)
(c) 대화 방식 · (d) 화면 위치	AI 추천안 수락

위임한 결정에 돌아온 보고

후보 6개에 실제 챗봇과 같은 프롬프트로 시험. 답이 있는 질문과 무관한 질문("이혼 재산분할 비율")을 던졌다. GLM 4.7 Flash는 빈 응답으로 탈락, 나머지 5개는 무관한 질문을 정확히 거절.

DeepSeek V4 Flash 추천: 질문당 약 2원(Claude 약 40원). "회사가 징계·전보를 다한 경우 책임이 부정되기도 한다"는 반대 사례를 스스로 짚었다. 단점은 16초로 느린 응답이라 스트리밍 출력으로 보완.

묻지 않은 한계도 먼저 고지

요약은 한 건에 190자라 법원이 왜 그렇게 판단했는지는 담겨 있지 않다. "347건 중 승소율은?" 같은 전체 통계 질문은 답하지 못한다(매번 20건만 보기 때문). 필요하면 상위 3~5건의 전문을 추가할 수 있다(질문당 2원 → 10원).

모르는 결정은 "추천하되 이유를 문서에 적어라"라는 조건으로 넘긴다. 근거가 문서로 남아 나중에 검토할 수 있다.

RAG 적용

시스템 프롬프트: 모델에게 주는 규칙 (AI가 적용한 것)

```
너는 한국의 '직장 내 괴롭힘' 관련 민사판결 347건만을 근거로 답하는 조수다.
사용자는 법률 전문가가 아니다. 반드시 지킬 규칙:
1. 아래 <판례>에 적힌 내용만 근거로 삼는다. 네가 따로 아는 법률지식이나
   일반론으로 보충하지 마라. <판례>에 없는 법조문·판례번호·금액을 지어내지 마라.
2. 답변의 각 주장 끝에 근거가 된 판례의 번호를 [1], [7]처럼 붙인다.
   반드시 <판례>에 실제로 있는 번호만 쓴다.
3. <판례>만으로 질문에 답할 수 없으면, 추측하지 말고 정확히 이렇게 답한다:
   "제가 가진 347건의 판결문으로는 답변드릴 수 없습니다" 그리고 이유를 한 줄로.
   일부만 답할 수 있으면 그 부분만 답하고, 나머지는 근거가 없다고 밝힌다.
4. 결론이 갈리는 쟁점이면 반대 방향의 사례도 함께 제시한다.
5. 한국어로, 법률용어는 풀어서 설명한다. 핵심부터 말하고 갈래를 잡아 정리한다.
6. 너는 변호사가 아니며 이 답변은 법률 자문이 아니다. 다만 매 답변마다
   그 사실을 반복해 적지는 마라 (화면에 이미 표시된다).
```

- **규칙 1·3**: 이 문장이 없으면 모델은 일반 법률지식으로 그럴듯하게 보충한다. 근거 밖 답변을 막는 장치
- **규칙 2**: 인용 번호는 검증 수단이다. 서버는 답변의 번호가 실제 20건 목록에 있는지 확인하고, 있는 번호만 근거 카드로 만든다. 모델이 붙인 번호를 그대로 믿지 않는다
- **규칙 4**: 법률 자료에서 예외를 함께 제시하게 한다
- **규칙 6**: 변호사법 제109조(비변호사의 법률사무 취급) 문제를 의식한 고지. 고지 문구는 화면에 고정 표시한다

RAG 적용

구현 완료 보고: “됩니다” 한 줄이 아니라 검증 결과로 받는다

1

답이 있는 질문

“회사에도 책임을 물을 수 있나요”: 책임이 인정된 경우와 부정된 경우로 나눠 답하고 근거 카드 13건, 인용 25개를 붙였다

2

범위 밖 질문

“이혼 재산분할”: 정해진 문구로 거절하고 근거 카드도 표시하지 않았다

3

애매한 질문

“녹음 파일이 몇 개나 필요한가요”: 구체적 숫자 기준은 답할 수 없다고 먼저 밝히고 판례에서 읽히는 경향만 설명

4

금액 질문

판례에 있는 금액만 인용. 요약의 위자료 금액과 카드의 전체 인용액(치료비 포함)은 성격이 다른 숫자임을 고지

구현 중 잡은 버그 2개

① 스트리밍 응답의 한글이 깨짐: 응답에 문자 인코딩 정보가 없어 라이브러리가 서유럽 문자로 읽었다 ② 모델이 [6, 13, 17]처럼 번호를 묶어 쓰는데 코드가 [6] 형태만 인식해 근거 카드가 적게 나왔다

답이 잘리는 경우

max_tokens를 1500으로 두었더니 답변이 문장 중간에서 끊겼다. 응답의 `finish_reason`이 `"length"`면 잘린 것이다. 화면에는 그럴듯하게 보이므로 모델을 바꿀 때마다 확인한다 (3000으로 올려 해결).

RAG 적용

완성된 챗봇 화면: 갈래를 나누고 근거 번호를 붙인다



- 인정 갈래와 부정 갈래를 나눠 답했다. 계획서에서 이 모델을 고른 이유였던 "반대 사례를 함께 짚는" 동작이 첫 질문에서 나왔다
- 본문의 번호 칩이나 아래 카드를 누르면 해당 판결문 전문이 열린다. 사용자가 근거를 바로 확인할 수 있는 구조다
- 답변은 2~4초 만에 첫 글자가 나오기 시작해 20~30초에 완성된다(스트리밍)
- 이어지는 대화: "그중에 회사 책임이 부정된 사례만 알려줘"라고 하면 내부적으로 "회사 책임이 부정된 직장 내 괴롭힘으로 인한 우울증 판례"로 다시 써서 검색한다. 다시 쓴 문장은 답변 위에 작게 표시된다

RAG 적용

실제 기록: 질문 한 번의 API 왕복 (키만 가리고 그대로)

```
// 전체 흐름: 실측 시간
[사용자 질문]
  → (1) 임베딩 API: 질문을 4,096개의 숫자로 변환            1.0초
  → [내 컴퓨터]   : 347건과 비교해 가까운 20건 선택         (API 호출 없음)
  → (2) 챗 API    : 요약 20건을 근거로 주고 답변 요청       12.0초
  → [답변 + 인용 번호]

POST https://openrouter.ai/api/v1/embeddings
Authorization: Bearer sk-or-v1-•••(비밀키)
{ "model": "qwen/qwen3-embedding-8b",
  "input": "직장 내 괴롭힘으로 우울증이 생겼을 때 회사에도 책임을 물을 수 있나요?" }
── 받은 것 ──
{ "data": [ { "embedding": [ -0.0036022…, -0.0106367…, 0.0236234… (총 4,096개) ] } ],
  "usage": { "prompt_tokens": 30, "cost": 0.0000003 }, "provider": "Nebius" }
```

순위	법원 · 사건번호	결과	유사도	맞은 쪽
1	수원지법 안산지원 2022가단87835	원고일부승	0.732	요약
2	수원지법 2024가단603107	원고일부승	0.707	요약
3	서울중앙지법 2022가단5334343	원고패	0.702	본문
4	수원지법 2021가단531133	원고일부승	0.691	요약
5	대구지법 경주지원 2019가단15840	원고일부승	0.685	요약

- 토큰 30개, 비용 0.0000003달러(약 0.00044원). 임베딩은 사실상 비용이 들지 않는다
- 판결마다 본문·요약 임베딩이 있어 총 694번 비교하고, 높은 쪽을 그 판결의 점수로 삼아 상위 20건을 고른다. 이 계산은 내 PC에서 한다
- 챗 요청 설정: `stream: true`는 답변을 쓰는 대로 조금씩 보내라는 뜻, `temperature: 0.2`는 답변을 되도록 일관되게 하라는 뜻(0에 가까울수록 매번 비슷)

RAG 적용

질문 한 번의 비용, 그리고 요청이 처리된 서버

단계	토큰	비용(원)
질문 임베딩	30	0.00044
(내 PC) 347건과 비교해 20건 선택	API 없음	0
답변 생성	입력 4,332 + 출력 761	1.19
합계	5,123	**약 1.19원**

1달러 = 1,450원 기준. 입력 토큰 대부분이 판례 요약 20건이다.

시각	모델	처리 서버(provider)
02:15	DeepSeek V4 Flash	Azure
02:15	Qwen3 Embedding 8B	DeepInfra
02:15	DeepSeek V4 Flash	StreamLake
02:13	DeepSeek V4 Flash	CoreWeave
02:13	Qwen3 Embedding 8B	Nebius
02:12	DeepSeek V4 Flash	GMICloud

OpenRouter Activity 화면의 실제 기록

법적으로 중요한 관찰: 우리는 모델 이름만 지정했는데, 그 모델을 어느 회사 서버에서 돌릴지는 OpenRouter가 가격·속도·혼잡도를 보고 그때그때 고른다. 같은 질문이 Azure, CoreWeave, StreamLake 등 다른 회사 서버로 갔다. 질문에 개인정보가 들어 있다면 **누구에게 처리를 맡겼는지(수탁자), 어느 나라로 이전되었는지**를 미리 특정하기 어렵다. 6강의 처리위탁(제26조)·국외 이전(제28조의8) 문제가 여기서 생긴다. OpenRouter는 provider를 지정하거나 데이터 보존 정책으로 거르는 설정을 제공한다.

RAG 적용

문서가 길다면: 청킹(chunking)

- 이번 사례는 비용을 고려해 판결문마다 **요약문 하나**만 컨텍스트에 실었다
- 업무 매뉴얼, 긴 보고서처럼 아주 긴 문서로 RAG를 만들 때 문서 전체를 컨텍스트에 넣으면 답변 품질에 도움이 되지 않고 비용만 오른다
- 그래서 문서를 쓸모 있는 단위로 잘라 **조각마다 임베딩**을 만들어 두고, 질문과 가까운 조각만 골라 컨텍스트에 넣는다. 이것이 청킹이다

①

고정 길이로 자르기

예: 500자씩, 앞뒤 50자를 겹쳐서. 구현이 쉽지만 문장·조항 중간이 잘릴 수 있다

②

구조 단위로 자르기

조문 단위, 판결문의 사실관계·법원의 판단 단위, 보고서의 절 단위. 문서에 대한 이해가 필요하다

③

조각에 맥락 붙이기

조각 앞에 "어느 문서의 어느 부분인지"를 한두 문장 덧붙여 임베딩한다. 조각만 떼어 놓으면 무엇에 관한 내용인지 잃기 때문

어디서 어떻게 자르는지가 RAG의 성능을 가른다. 이것은 기술 문제이기 전에 **문서를 아는 사람이 정할 일**이다. 판결문이라면 사실관계와 법리를 나누어 자르는 식이다. 잘못 자르면 정답이 들어 있는 조각이 검색되지 않는다.

RAG 적용

대안: 검색 없이 통째로 읽히면 안 되나 (Long Context)

	RAG (검색 후 로딩)	Long Context (통째로 로딩)	하이브리드
컨텍스트에 싣는 것	검색된 요약 20건 (약 3천 토큰)	요지 347건 전부 (약 4~5만 토큰)	요지 전부 + 모델이 고른 판결의 전문을 도구로 읽어 추가
규모	수만~수백만 건	모델의 컨텍스트 창에 들어가는 만큼	요지가 컨텍스트에 들어가는 만큼
구현	임베딩 + 벡터 DB + 백엔드	파일 첨부 + 프롬프트	파일 읽기 도구 필요 (Cowork, Claude Code, 도구 호출 API)
1회 비용	낮음	높음 (매 질문 전체 과금)	높음 + 도구 왕복만큼 느림
실패 모드	검색이 놓치면 답도 틀림	창을 넘거나, 긴 문맥 중간의 정보를 놓침	모델이 요지만 보고 답하고 끝내는 경우
어울리는 일	반복 질문, 서비스, 대규모	혼자 하는 전체 경향 분석	요지는 다 보되 전문은 골라 읽는 연구 작업

- 사용 전 점검: 자료의 토큰 수를 추정한다(한국어는 대략 글자 수 × 0.7~1). 모델의 컨텍스트 창과 비교해 절반을 넘으면 여유가 없다. 창의 명목 길이가 실제 독해력을 보장하지는 않는다
- Cowork·Claude Code가 일하는 방식이 하이브리드다. 목록을 훑고 필요한 파일만 읽는다. 이 절차를 반복해 쓰려면 **스킬**로 저장한다(3강 §9, 과제 (i))

RAG 적용

무엇을 싣는가, 그리고 내 자료가 모델의 컨텍스트 창에 들어가는가

RAG: 골라서 싣기

지시 제공 문서만 근거로 답하라…
검색된 판례 요약 20건
질문 동료들의 따돌림도…?

약 3천 토큰. 질문마다 검색으로 새로 고름

Long Context: 통째로 싣기

지시 첨부 자료 전체를 읽고 답하라…
판결 요지 347건 전부
질문 동료들의 따돌림도…?

약 4~5만 토큰. 질문마다 전체를 다시 실음

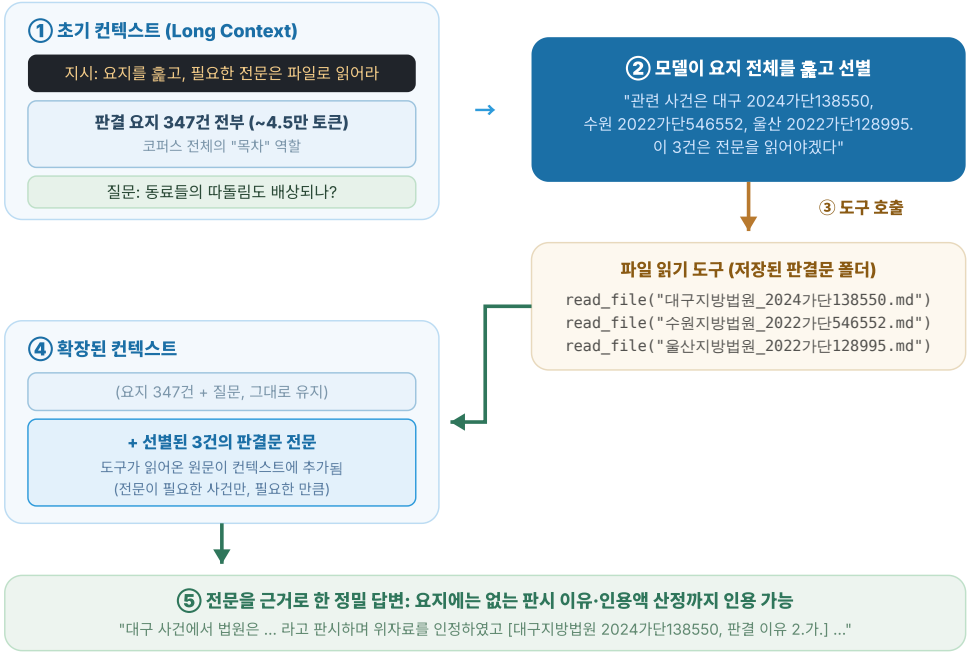
- Long Context는 백엔드도 임베딩도 필요 없다. 요지 CSV를 Chat·Cowork에 첨부하고 "전부 읽고 답해줘"라고 하면 된다. 혼자 하는 연구라면 가장 단순한 선택지다
- 점검 순서: ① 자료의 토큰 수 추정(한국어는 글자 수 × 0.7~1, 요지 347건 약 6.5만 자 ≈ 4~5만 토큰) ② 모델의 창과 비교, 절반을 넘으면 여유 없음 ③ 명목 길이가 실제 독해력을 보장하지 않으므로 중요한 지시·자료는 앞이나 뒤에

구분	모델	컨텍스트 창	요지 347건 수용
해외 상용	Google Gemini 3.1 Pro	100만	✅ 여유
	OpenAI GPT-5.5	100만	✅ 여유
	Claude Opus 4.8 / Sonnet 4.6	100만	✅ 여유
중국 오픈소스	DeepSeek V4	100만	✅ 여유
	Qwen3.5-Plus	100만	✅ 여유
	Kimi K2.5	26만	✅ 가능
국내	LG K-EXAONE (236B)	26만	✅ 가능
	네이버 HyperCLOVA X SEED Think	3.2만	⚠️ 들어가지 않음

단위: 토큰. 2026년 공개 정보 기준(부트캠프 자료)이며 버전에 따라 달라진다. 사용 전 공식 문서로 확인한다. 같은 자료라도 모델에 따라 Long Context 방식이 되기도, 안 되기도 한다. 들어가지 않으면 요약본을 쓰거나, 나눠 묻거나, RAG로 전환한다.

RAG 적용

하이브리드: 요지는 통째로 싣고, 전문은 골라 읽는다



💬 프롬프트

연동 폴더에 판결 요지 CSV(347건)와 판결문 전문 파일들(rulings_md/)이 있어.
질문: 동료 직원들의 따돌림에 대해 손해배상이 인정된 사건과 기각된 사건의 갈림길은 무엇이었나?
먼저 요지 전체를 훑어서 관련 사건을 고르고, 고른 사건은 반드시 전문 파일을 읽은 다음,
전문을 근거로 사건번호와 판시 대목을 인용해서 답해줘. 요지만 보고 답하지 마.

복사

SECTION 4

RAG의 한계: 근거를 붙여도 여전히 틀린다

RAG는 환각을 줄이지만 없애지 못한다. 검색 단계와 생성 단계 각각에서 실패가 일어나고, 검색 대상 자료가 공격받거나 개인정보를 드러낼 수도 있다. 논문과 실제 사고 사례로 확인한다.

1 Claude Code 시작하기
2 검색엔진 구현
3 RAG 적용
4 RAG의 한계
5 Vercel 배포
6 인증과 권한
7 로깅: Supabase
8 자동화 대상 고르기
9 정리와 과제

RAG의 한계

틀렸다면 어디서 틀렸나: 검색 실패와 생성 실패를 나눈다

검색 실패: 정답이 담긴 판결이 top-20에 들지 못했다. 모델은 받은 20건 안에서 최선을 다했지만 애초에 재료가 틀렸다.

생성 실패: 정답 판결이 top-20에 있었는데 답이 틀렸거나, 인용한 판결이 그 문장을 뒷받침하지 않는다.

Barnett 외, Seven Failure Points When Engineering a RAG System (CAIN 2024): 연구·교육·생의학 분야 RAG 구축 사례에서 정리한 7가지 실패 지점

단계	실패 지점	판결문 챗봇에서는
데이터	FP1 Missing Content: 자료에 답이 없는데 "모른다" 대신 그럴듯한 답을 만든다	347건에 없는 쟁점(예: 육아휴직 불이익)을 물었는데 일반론으로 답함
검색	FP2 Missed the Top Ranked Documents: 답이 있는 문서가 상위 k건에 들지 못한다	정답 판결이 21위
검색 후처리	FP3 Not in Context: 검색은 되었지만 모델에게 전달되는 문맥에서 빠진다	길이 제한으로 잘린 요약
생성	FP4 Not Extracted · FP5 Wrong Format · FP6 Incorrect Specificity · FP7 Incomplete	문맥에 있는데 놓침, 지나치게 일반적인 답, 일부만 답함

- 디버깅 순서: 오답을 발견하면 **먼저 top-20 목록에 정답 판결이 있었는지** 본다. 없으면 검색 문제, 있으면 생성 문제다(§3에서 본 원칙)
- 저자들의 결론: RAG 시스템의 검증은 운영 중에만 실제로 가능하고, 견고성은 설계 단계에서 정해지지 않고 운영하면서 높아진다

RAG의 한계 · 실증 연구

법률 RAG 실측: 상용 도구도 17~33% 환각

기준선: RAG 없는 LLM

- Dahl 외, *Large Legal Fictions* (Journal of Legal Analysis 2024): 미국 연방법원 사건에 대한 검증 가능한 질문에서 환각률 **58%(ChatGPT 4)~88%(Llama 2)**
- 사용자의 잘못된 법적 전제를 바로잡지 못하고 받아들이는 경향, 자신이 환각을 일으키는지 잘 예측하지 못하는 경향도 관찰

RAG를 붙인 상용 법률 도구

- Magesh 외, *Hallucination-Free?* (Journal of Empirical Legal Studies 2025, Stanford): 사전 등록한 200개 이상의 법률 질의

도구 (2024. 5. 시험)	정확	환각
Lexis+ AI	65%	약 17%
Westlaw AI-Assisted Research	42%	약 33%
Ask Practical Law AI	60% 이상 불완전한 답(거절 포함)	

환각의 정의가 핵심이다. 이 연구는 답이 **틀린 경우**뿐 아니라, 법을 맞게 서술했어도 **인용한 출처가 그 주장을 뒷받침하지 않는 경우(misgrounded)**도 환각으로 센다. "인용 번호가 붙어 있으니 믿을 수 있다"는 가정이 성립하지 않는다는 뜻이다.

- 벤더들은 RAG로 환각을 "제거"했다거나 "환각 없는" 인용을 보장한다고 홍보했다. 연구진은 이 주장이 과장되었다고 결론지었다(Thomson Reuters는 내부 테스트 정확도 약 90%라고 반론)
- 두 연구는 같은 연구팀이지만 질의 세트와 평가 기준이 달라 수치를 직접 빼서 비교하지 않는다. 말할 수 있는 것은 "RAG로 줄지만 0이 되지 않는다"까지다
- 판결문 데이터에는 상급심에서 파기된 판결이 섞이기 쉽다. 검색이 "관련 있는" 판결을 찾아도 그것이 현재 유효한 법리인지는 별개 문제다

RAG의 한계 · 생성 단계

생성 단계 실패: 틀린 근거를 따르고, 엇갈리면 원래 알던 쪽을 고른다

1

틀린 검색 결과를 그대로 따른다

- **ClashEval** (Wu·Wu·Zou, NeurIPS 2024): 약물 용량, 기록, 지명 등 1,200개 이상의 질문에서 검색 문서에 오류를 심었다. GPT-4o 등 6개 모델이 원래 맞게 알던 답을 버리고 틀린 검색 내용을 채택한 경우가 **60%를 넘었다**
- RAG의 답은 검색 자료의 품질에 묶인다. 요약문의 오기, 잘못된 금액, 파기된 판결이 그대로 답변이 된다

2

자료가 엇갈리면 원래 알던 쪽을 고른다

- **Adaptive Chameleon or Stubborn Sloth** (Xie 외, ICLR 2024): 외부 근거가 하나이고 일관되면 모델은 자기 지식과 달라도 받아들인다. 그러나 근거 안에 서로 다른 내용이 섞여 있으면 자기 지식과 맞는 쪽을 강하게 고른다(확증 편향)
- top-20 안에 결론이 다른 판결들이 섞이면, 모델은 “더 많은” 쪽이 아니라 “원래 알던” 쪽을 고를 수 있다

RAG의 한계 · 검색 단계

검색 단계 실패 (1): 조각은 맥락을 잃고, 임베딩에는 이론적 한계가 있다

Anthropic, Introducing Contextual Retrieval (2024. 9.)

- 실패 원인: 문서를 조각으로 자르면 맥락이 사라진다. 공시 문서의 조각 “The company's revenue grew by 3% over the previous quarter.”에는 회사명도 기간도 없어 “ACME의 2023년 2분기 매출 성장률” 질의로 찾기 어렵다

방법	top-20 검색 실패율
일반 임베딩	5.7%
조각에 맥락 문장을 붙여 임베딩	3.7% (35% 감소)
+ 키워드 검색(BM25) 병행	2.9% (49% 감소)
+ 재순위화(reranking)	1.9% (67% 감소)

최선의 조합에서도 실패가 남는다. 수치는 회사가 고른 데이터셋의 평균이고, 쟁점이 미묘한 법률 문서에서는 다를 수 있다.

Weller 외, On the Theoretical Limitations of Embedding-Based Retrieval (Google DeepMind, 2025 · ICLR 2026)

- 문서 하나를 벡터 하나로 나타내는 방식은, top-k로 돌려줄 수 있는 **문서 조합의 수가 임베딩 차원에 의해 상한**이 정해진다. k=2인 단순한 경우에도, 시험 데이터에 직접 맞춰 학습한 임베딩에서도 한계가 나타난다
- LIMIT 데이터셋: “○○는 △△를 좋아한다” 같은 단순한 문장과 “△△를 좋아하는 사람은?” 질의. 문서 5만 건 버전에서 최상위 임베딩 모델들도 **recall@100이 20% 미만**. 같은 과제에서 키워드 기반 BM25는 훨씬 나았다

> 더 좋은 임베딩 모델로 바꾸면 해결된다는 기대에는 한계가 있다. 질의가 여러 조건의 조합을 요구할수록(예: “2020년 이후, 근로자 패소, 연장근로수당 쟁점”) 벡터 하나로는 다 표현하지 못한다. 조건 검색은 메타데이터 필터나 데이터베이스 질의로 처리한다.

RAG의 한계 · 검색 단계

검색 단계 실패 (2): 부정어, 숫자·정확한 용어, 그리고 긴 문맥

¬

부정어를 구별하지 못한다

NevIR (Weller 외, EACL 2024): 부정 표현 유무만 다른 문서 쌍에서 두 질의 모두의 순위를 맞히게 했더니, 대부분의 최신 검색 모델이 **무작위 순위와 비슷하거나 그보다 나빴다**. "해고가 정당하다"와 "해고가 정당하지 않다", 책임을 인정한 판결과 인정하지 않은 판결은 임베딩 공간에서 매우 가깝다

#

숫자·조문 번호·정확한 용어

금액, 날짜, 사건번호, "제76조의2" 같은 정확한 일치가 필요한 검색에는 의미 검색보다 키워드 검색이 적합하다. 실무에서는 두 방식을 섞는다(하이브리드 검색). Anthropic이 BM25를 병행하는 이유로 든 것도 정확한 용어 일치다. 다만 이 약점을 수치로 측정한 대표 연구는 아직 확인하지 못했다

⋯

많이 넣을수록 중간을 놓친다

Lost in the Middle (Liu 외, TACL 2024): 정답이 입력의 처음이나 끝에 있을 때 가장 잘 찾고 중간에 있을 때 크게 떨어지는 U자형. GPT-3.5는 문서 없이 56.1%, 정답 문서 하나만 주면 88.3%였는데, **문서 20개** 중 가운데에 정답을 두면 문서를 주지 않은 경우보다 낮아지기도 했다. 우리 챗봇과 같은 20건 설정이다

Context Rot (Chroma 기술 보고서, 2025): 18개 모델 모두 입력이 길어질수록 성능이 떨어졌다. 단순한 과제에서도 그랬고, 정답과 비슷한 방해 문장이 있을 때 더 떨어졌다. top-20에는 정답과 비슷하지만 결론이 다른 판결이 섞이기 마련이다.

데모 ①로 돌아가 보면: "괴롭힘을 인정하지 않은 사건"은 키워드 검색의 실패 사례였다. 임베딩 검색은 이 질의에서 성공했지만(§2), 부정어 처리가 항상 성공한다는 보장은 없다. 결론이 중요한 질의는 결과 목록의 소송결과 칸을 직접 확인한다.

RAG의 한계 · 보안과 프라이버시

보안과 프라이버시: 검색 자료가 공격 경로가 되고, 벡터도 원문을 드러낸다

지식 DB 오염 (PoisonedRAG)

Zou 외, USENIX Security 2025. 수백만 건 규모의 지식 DB에 목표 질문당 **악성 텍스트 5개**를 넣어 공격자가 고른 답을 내게 하는 공격의 성공률이 **90%**. 저자들이 검토한 기존 방어 기법으로는 충분히 막지 못했다. 사용자 업로드 문서, 웹에서 수집한 자료를 DB에 섞기 시작하면 이 위험이 생긴다

간접 프롬프트 주입

Greshake 외, AISec 2023. 공격자가 LLM이 나중에 읽을 웹페이지·문서·이메일에 지시문을 숨겨 두면, 사용자가 아닌 제3자가 모델의 행동을 바꿀 수 있다. Bing Chat 등 실제 시스템에서 시연했다. RAG에서 검색된 문서는 "데이터"이지만 모델에게는 "지시"로도 읽힌다. 데이터와 명령의 경계가 구조적으로 없다

임베딩 역추적

Morris 외, *Text Embeddings Reveal (Almost) As Much As Text*, EMNLP 2023. 임베딩만 보고 원문을 반복 수정하며 복원하는 방법으로 32토큰 입력의 **92%를 정확히 그대로** 복원했다. 임상 기록 임베딩에서 환자 성명 같은 개인정보도 복원할 수 있음을 보였다

법적 질문 (6강과 연결): 개인정보가 담긴 문서를 임베딩한 벡터 DB는 개인정보인가? 원문만 지우고 벡터를 남기면 파기한 것인가? 위 연구에 비추면 임베딩도 원문을 상당 부분 복원할 수 있는 정보다. 벡터 DB에도 접근 통제, 안전조치, 파기 의무를 같이 적용하는 것이 안전한 해석이다.

RAG의 한계 · 실제 사고

실제 사고와 책임: “AI가 한 말”은 면책 사유가 되지 않았다

사례	무슨 일이 있었나	결과와 시사점
Moffatt v. Air Canada (2024 BCCRT 149, 2024. 2.)	조모상을 당한 승객에게 웹사이트 챗봇이 “일반 운임으로 예약한 뒤 사별 할인을 소급 신청할 수 있다”고 안내. 실제 정책은 소급 신청 불가	항공사는 “챗봇은 스스로 책임지는 별개 주체”라고 항변했으나 배척. 웹사이트 정보는 정적 페이지든 챗봇이든 회사가 책임진다(부주의한 허위 진술). 배상 CAD 650.88, 이자·수수료 포함 812.02
뉴욕시 MyCity 챗봇 (2024. 3. 보도)	시 공식 자료를 바탕으로 사업자에게 규정을 안내하는 공공 챗봇이 “주거 바우처 세입자를 거절해도 된다”, “직원 팁 일부를 가져가도 된다”는 위법한 답을 함. 같은 질문에 답이 달라지기도 함	당시 시장은 운영을 유지. 2026. 1. 새 시장이 “기능적으로 쓸 수 없다”며 폐지 계획 발표. 공식 자료를 근거로 해도 답의 적법성은 보장되지 않는다
Cursor 고객지원 봇 “Sam” (2025. 4.)	여러 기기에서 로그아웃되는 문제 문의에 “구독당 기기 1대만 허용하는 보안 정책”이라고 답함. 그런 정책은 없었고 Sam은 AI였다	공동창업자가 공개 사과, 이후 AI 응답임을 표시하기로 함. 자료에 답이 없을 때(FP1) “모른다” 대신 정책을 지어낸 사례
Deloitte 호주 보고서 (2025)	정부에 제출한 237쪽 보고서에 존재하지 않는 참고문헌과 지어낸 판결 인용. 수정본에서 GPT-4o 기반 도구 사용을 밝힘	계약(약 AUD 44만)의 마지막 분할금 환불. RAG 사례는 아니지만 검증하지 않은 인용의 전문가 책임 사례
국내 법원 (2025~2026)	존재하지 않는 판례가 인용된 준비서면·의견서 제출이 여러 차례 보도	법원행정처 TF 대응 방안(소송비용 부담 등). 2026. 2.부터 사법정보공개포털에서 사건번호 실재 여부 확인 가능. 다만 번호가 실재해도 판시 내용을 다르게 인용한 경우(misgrounded)는 걸러지지 않는다

RAG의 한계 · 직접 확인

우리 챗봇으로 직접 확인하기: 실패를 일부러 찾아본다

1

자료에 없는 질문

347건에 없는 쟁점을 묻는다(예: “육아휴직 후 복귀 거부로 인한 손해배상”). 정해진 거절 문구가 나오는지, 일반론으로 그럴듯하게 답하는지 본다 (FP1)

2

부정·결론을 묻는 질문

“회사 책임을 **부정한** 판결만”처럼 결론을 지정한다. 근거 카드의 소송결과가 실제로 그 결론인지 확인한다

3

숫자를 묻는 질문

“위자료 3천만 원 이상 인정된 사건”. 금액 조건은 임베딩이 잘 다루지 못한다. 답변의 금액을 카드·전문과 대조한다

4

인용 대조

답변의 문장 하나를 골라, 붙은 번호의 판결 전문을 열어 그 문장을 실제로 뒷받침하는지 읽는다. 번호가 있는 것과 뒷받침하는 것은 다르다(misgrounded)

💬 프롬프트

복사

```
챗봇 디버깅용 기능을 추가해줘. 관리자 화면에서 각 질문에 대해
(1) 검색된 20건의 순위·사건번호·유사도·소송결과를 함께 보여주고
(2) 답변에 인용된 번호마다 해당 요약의 첫 문장을 나란히 표시해서
내가 인용이 문장을 뒷받침하는지 눈으로 대조할 수 있게 해줘.
틀린 답을 발견하면 "검색 실패"와 "생성 실패" 중 무엇인지 표시해 기록할 수 있게 해줘.
```

이렇게 모은 실패 사례가 평가 세트가 된다. 과제 발표의 “신뢰성” 부분에 쓴다.

RAG의 한계

완화책, 그리고 그래도 남는 한계

단계	완화책	남는 한계
데이터	출처 통제(누가 무엇을 넣을 수 있나), 파기·변경된 판결 표시, 요약의 사실 검증, 개인정보 제거 후 임베딩	자료 자체의 오류와 시의성은 운영자가 계속 관리해야 한다
검색	키워드(BM25) + 임베딩 하이브리드, 조각에 사건명·법원·쟁점 붙이기, 재순위화, 소송결과·연도 같은 조건은 메타데이터 필터로	최선의 조합에서도 검색 실패가 남는다(Anthropic 실측 1.9%). 임베딩의 이론적 한계
생성	문맥 수 줄이기(재순위화 후 상위 5건), 가장 관련 높은 자료를 앞쪽에, "자료에 없으면 모른다" 규칙, 인용 번호 검증(§3), 인용 문장과 원문 대조 검사	상용 법률 도구도 환각 17% 이상. 인용이 붙어도 뒷받침하지 않을 수 있다
보안	검색 문서 안의 지시문을 따르지 않도록 하는 지시, 업로드 자료 검토, 벡터 DB 접근 통제	간접 프롬프트 주입을 완전히 막는 방법은 아직 없다
운영	로그(§7)로 실패 사례를 모으고 평가 세트로 측정, 화면에 한계 고지, 사람이 최종 확인	운영자 책임은 남는다(Air Canada)

RAG는 "근거를 확인할 수 있게 해 주는 장치"이지 "근거가 맞음을 보장하는 장치"가 아니다. 최종 확인은 사람이 원문으로 한다

SECTION 5

Vercel 배포: 내 PC의 서비스를 공개 URL로

로컬 서버로 돌던 검색·챗봇 웹앱을 Vercel의 서버리스 함수로 옮겨, 누구나 접속하는 URL을 받는다. 비밀은 코드가 아니라 서버의 환경변수에 둔다.

1 Claude Code 시작하기
2 검색엔진 구현
3 RAG 적용
4 RAG의 한계
5 Vercel 배포
6 인증과 권한
7 로깅: Supabase
8 자동화 대상 고르기
9 정리와 과제

VERCEL 배포

지금까지의 한계: 왜 나만 쓸 수 있었나



PC를 끄면 서비스도 꺼진다

노트북을 닫으면 검색기도 접속할 수 없다

주소가 localhost

127.0.0.1은 "이 컴퓨터 자신"이라는 뜻이다. 남에게 보낼 수 있는 주소가 아니다

동시 사용자 = 나 혼자

동료에게 "이거 써 봐"라고 보낼 방법이 없다

키를 가진 백엔드를 내 PC에서 클라우드로 옮긴다. 코드는 거의 그대로이고 실행 위치만 바뀐다

VERCEL 배포

Vercel과 서버리스 함수: push하면 배포된다

	GitHub Pages (3강)	Vercel (오늘)
올릴 수 있는 것	정적 파일 (HTML·CSS·JS)	정적 파일 + 서버에서 실행되는 코드
비밀 보관	불가능. 모든 파일이 공개	가능. 환경변수에 저장
배포 방법	저장소에 push	저장소에 push (같다)
무료 범위	공개 저장소면 무제한	Hobby 플랜 (개인·비상업 프로젝트)
주소	아이디.github.io/저장소	프로젝트명.vercel.app

	로컬 서버 (§2·3)	Vercel 서버리스 함수
실행	터미널에서 직접 실행	`api/` 폴더에 파일을 넣으면 끝
살아 있는 시간	내가 켜 둔 동안	요청이 올 때만 실행되고 종료
주소	localhost:8000/search	내앱.vercel.app/api/search
키	내 PC의 키 파일	Vercel 환경변수 설정 화면

- **서버리스(serverless)**는 서버가 없다는 뜻이 아니라 **서버를 내가 관리하지 않는다**는 뜻이다. 서버는 Vercel의 데이터센터에 있고, 나는 코드만 올린다
- 요청이 올 때만 실행되므로 실행된 시간만큼만 자원을 쓴다. 개인 프로젝트나 MVP 수준의 트래픽이면 무료 한도 안에서 운영된다
- Hobby 플랜은 개인·비상업 용도로 제한된다. 회사 업무용으로 계속 쓰려면 약관과 유료 플랜을 확인한다

VERCEL 배포

변환 지시: AI가 먼저 위험을 지적했다

"Vercel에 배포할 수 있게 바꿔 달라"고 지시하자, Claude Code는 바로 바꾸지 않고 두 가지 문제를 먼저 제기했다.

① 판결문 347건이 인터넷에 공개된다
저장소를 비공개로 한 이유가 바로 이것이었다(DECISIONS.md의 D-12 결정). Vercel에 올리면 URL을 아는 누구나 볼 수 있다. 무료 플랜에는 비밀번호 보호 기능이 없다.

② API 키가 무방비로 노출된 것과 같다
공개 URL 뒤의 챗봇은 누구나 무제한으로 질문할 수 있고, 비용은 전부 내 OpenRouter 잔액에서 나간다. 질문당 1.2원이라도 자동화된 요청 수만 건이면 큰 금액이 된다.

💬 프롬프트

현재 웹앱을 vercel에 배포할 수 있도록 바꿔줘. 판결문 공개되어도 무방함.
openrouter credit 부담할 것이야. 나중에 인증 기능 추가할 예정이라서
지금은 상관 없음.

복사

- §2에서 남긴 결정 기록을 AI가 스스로 찾아 새 지시와 충돌한다고 지적했다. 문서화의 효과다
- 둘 다 기술 문제가 아니라 **사람이 내릴 결정**이다. 위험을 듣고 감수 여부를 정하는 것은 사용자의 몫이고, 그 결정(공개 무방, 비용 부담, 인증은 §6에서)도 기록으로 남는다
- 업무 데이터라면 답이 달라진다. "공개되어도 무방함"이라고 말할 수 있는지부터 따져 본다

VERCEL 배포

배포 4단계

1

Vercel 가입

vercel.com → Sign Up → **Hobby** → **Continue with GitHub**. 첫 화면의 Import Git Repository에서 **Install GitHub**으로 저장소 접근을 허용한다

2

코드 변환

앞 장의 지시로 로컬 서버 코드를 서버리스 함수 형식으로 바꾼다. 키는 파일 대신 **환경변수**에서 읽도록 바뀐다

3

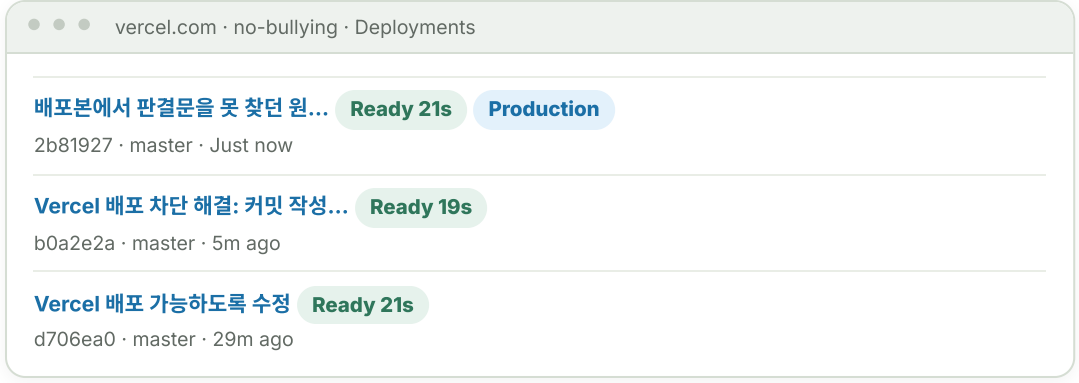
push → Import

변경을 commit·push. Vercel 대시보드 → Add New… → Project → 저장소 옆 **Import**

4

New Project 화면

Application Preset(이 빌드는 Flask, 모르면 Claude Code에 질문) 확인. **Environment Variables**에 `OPENROUTER_API_KEY`와 키 값 입력 → Deploy



- 1~2분 뒤 `프로젝트명.vercel.app` 주소가 나온다. 다른 사람의 휴대폰에서 열어 검색이 되면 배포에 성공한 것이다. 내 PC는 꺼도 된다
- 이후에는 **push가 곧 배포**다. Vercel이 새 커밋을 감지해 빌드(라이브러리 설치, 함수 묶기)한 뒤 교체한다
- 빌드가 실패하면 이전 버전이 그대로 서비스된다. 고치다 실수해도 사이트가 멈추지 않는다
- 행 하나가 배포 한 번이고 이름은 커밋 메시지다. 커밋 번호로 어느 코드가 올라가 있는지 맞춰 볼 수 있다

VERCEL 배포

실제 배포 화면: New Project, 그리고 환경변수 관리

vercel.com/new · New Project

Importing from GitHub · ● kaist-itm-aibootcamp/no-bullying ↪ master

Vercel Team KAIST-ITM · Hobby

Project Name no-bullying

Application Preset Flask ⌄

Root Directory ./ (Edit)

› Build and Output Settings

⌄ **Environment Variables**

Key `OPENROUTER_API_KEY`

Value (3강에서 발급한 내 키 붙여넣기) 👁 🔒

Environments Production and Preview

Import .env or paste the .env contents · + Add More

Deploy

no-bullying · Settings · Environment Variables

🔒 **PASSWORD_HASH** Sensitive Production
Added 17m ago

🔒 **ALLOWED_EMAILS** Sensitive Production
Added 40m ago

🔒 **SMTP_APP_PASSWORD** Sensitive Production
Added 49m ago

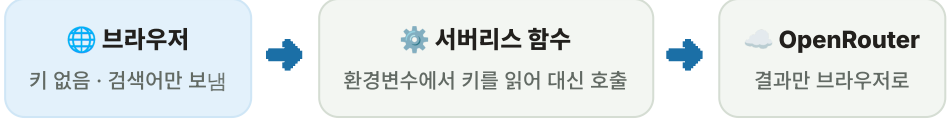
🔒 **SMTP_USER** Sensitive Production
Added 49m ago

- Team·Project Name은 기본값 그대로. 이름이 이미 쓰이고 있으면 Vercel이 접미사를 붙인다(no-bullying-**gamma**)
- Environment Variables를 빠뜨리면 배포는 되지만 검색할 때 서버가 키를 찾지 못해 실패한다
- 배포 후의 관리: 프로젝트 → 왼쪽 메뉴 **Environment Variables**에서 추가·수정·삭제. "Sensitive"는 값이 가려져 저장된다는 표시. 바꾼 뒤에는 재배포해야 반영된다
- Note 칸("Where to rotate, or who to contact")에 키를 어디서 재발급하는지 적어 두면 유출 시 대응이 빠르다

VERCEL 배포

핵심: 비밀은 코드가 아니라 서버의 환경변수에

환경변수	코드 바깥, 서버의 설정 공간에 저장하는 이름=값 쌍. 코드에는 "OPENROUTER_API_KEY라는 이름의 값을 읽어라"만 적혀 있고, 실제 값은 Vercel 설정 화면에만 있다
코드에 넣지 않는 이유	코드는 GitHub에 올라간다. 공개 저장소면 누구나, 비공개라도 저장소 접근자 전원이 키를 본다. 노출된 키를 자동으로 수집해 도용하는 프로그램이 상시 동작한다
브라우저에 넣지 않는 이유	프런트엔드 JS에 넣은 값은 개발자도구의 Network·Sources 탭에서 누구나 볼 수 있다. 브라우저에 전달된 값은 공개된 값이다



로컬 개발용 `.env` 파일: 내 PC에서는 같은 값을 프로젝트 폴더의 `.env` 파일에 `이름=값` 으로 둔다. 코드는 내 PC에서는 .env에서, Vercel에서는 설정 화면에서 같은 이름을 읽는다. .env는 비밀 모음이므로 `.gitignore` 에 들어 있는지 반드시 확인한다.

오늘 나오는 비밀(OpenRouter 키, Gmail 앱 비밀번호, Supabase 키, 비밀번호 해시)은 모두 같은 자리에 둔다

SECTION 6

인증과 권한: 허용된 사람만, 허용된 일만

URL이 생기는 순간 무엇이 달라지는지 정리한다. 가장 간단한 비밀번호 잠금(해시 + 환경변수)에서 출발해, 이메일 6자리 인증코드로 명단의 사람만 쓰게 하고, 그 밖의 인증 기법과 권한 설계를 개관한다.

1 Claude Code 시작하기
2 검색엔진 구현
3 RAG 적용
4 RAG의 한계
5 Vercel 배포
6 인증과 권한
7 로깅: Supabase
8 자동화 대상 고르기
9 정리와 과제

인증과 권한

URL이 생기는 순간 달라지는 세 가지

모르는 방문자

초대한 사람만 오지 않는다. 흔한 주소 패턴을 자동으로 두드리는 봇은 배포 직후부터 접속한다. vercel.app 주소는 생각보다 쉽게 퍼진다

비용은 내 계정으로

서비스가 부르는 API(임베딩·LLM) 요금은 전부 내 키로 청구된다. 누가 얼마나 쓰게 할지 내가 정해야 한다. 키에 사용 한도를 걸어 두는 것이 1차 안전장치다

데이터의 책임

서비스에 담긴 데이터와 서비스가 수집하는 데이터(질문, 이메일)의 관리 책임이 나에게 생긴다. 개인정보 보호법상 개인정보처리자가 될 수 있다

인증과 권한

가장 간단한 잠금의 재료: 해시(hash)란 무엇인가

- **해시 함수**는 어떤 길이의 입력이든 받아서 **정해진 길이의 문자열**(해시값)을 내놓는 함수다. SHA-256은 입력이 한 글자든 책 한 권이든 항상 64자리 16진수(256비트)를 낸다

```
# 실제 SHA-256 계산 결과
kaist2026  →  5391049cb77f42e054372ef575adc8c88a8000ef63fcd48203fa10fd4d4c3d4e
kaist2027  →  ff88f3baea2d5fb57236735d592127c1733d4915540310728193df425beb7193   ← 숫자 하나만 바꿈
Kaist2026  →  9dce81ff753e1363ad924a7a7ecceaf5508258cf322aa8702f71a00171676ca9   ← 대문자 하나만 바꿈
```

1

같은 입력, 같은 출력

언제 어디서 계산해도 kaist2026의 해시는 같다. 그래서 "입력값을 해시해 저장된 해시와 비교"하는 방식으로 비밀번호를 확인할 수 있다

2

되돌릴 수 없다

해시값에서 원래 입력을 계산해 내는 방법이 없다(일방향). 그래서 해시값이 노출되어도 원래 비밀번호가 바로 드러나지 않는다

3

조금만 달라도 전혀 다르다

한 글자만 바뀌어도 해시값이 완전히 달라진다. 비슷한 비밀번호인지조차 해시값으로는 알 수 없다

4

같은 해시를 만드는 다른 입력을 찾기 어렵다

현재 기술로 SHA-256의 해시값이 같은 서로 다른 두 입력을 찾아낸 사례가 없다(충돌 저항성)

인증과 권한

해시만으로 충분한가: 약점과 보완, 그리고 법적 의무

약점: 되돌릴 수는 없지만 "추측해서 맞춰 볼" 수는 있다

- **무차별 대입**: 후보 비밀번호를 하나씩 해시해서 같은 값이 나오는지 본다. SHA-256은 빠르게 계산되도록 설계되어 있어 일반 GPU로도 초당 수십억 번 시도할 수 있다
- **미리 계산한 표**: `123456`의 해시는 누가 계산해도 `8d969eef…6c92`다. 흔한 비밀번호의 해시를 미리 계산해 둔 표가 공개되어 있어, 짧거나 흔한 비밀번호는 해시값만 보고 바로 찾아진다

보완

- **솔트(salt)**: 사용자마다 다른 임의 문자열을 붙여서 해시한다. 같은 123456이라도 `k7Qz`를 붙이면 `756220f0…1598`, `Xp2m`을 붙이면 `596e4d6c…8400`. 미리 계산한 표가 쓸모없어진다
- **느린 해시**: bcrypt, scrypt, Argon2처럼 일부러 계산을 느리게(수십~수백 밀리초) 만든 함수를 쓴다. 정상 로그인에는 문제가 없지만 수십억 번 시도는 사실상 불가능해진다
- **긴 비밀번호**: 문장형으로 길게 정하면 시도해야 할 경우의 수가 급격히 늘어난다

상황	적정 수준
나 혼자 쓰는 웹앱의 사이트 비밀번호 (오늘의 지름길)	SHA-256 해시 + 환경변수 + 긴 문장형 비밀번호
여러 사람의 회원 비밀번호를 저장하는 서비스	솔트 + 느린 해시(bcrypt, Argon2). 직접 구현하지 말고 검증된 라이브러리나 인증 서비스 사용

법적 의무: 개인정보보호위원회 「개인정보의 안전성 확보조치 기준」(고시)은 개인정보처리자가 비밀번호를 저장할 때 **복호화되지 않도록 일방향 암호화**해 저장하도록 정한다. 다른 사람의 비밀번호를 받는 서비스를 만들면 이 의무가 적용된다. 개인정보 보호법 제29조(안전조치의무)의 구체화다.

해시와 암호화의 차이: 암호화는 키가 있으면 원래대로 되돌릴 수 있다(양방향). 해시는 누구도 되돌릴 수 없다(일방향). 비밀번호는 원래 값을 다시 볼 필요가 없으므로 해시로 저장한다. 반대로 API 키처럼 서버가 원래 값을 써야 하는 비밀은 해시할 수 없어서 환경변수에 원문을 둔다.

인증과 권한

가장 간단한 잠금: 비밀번호 해시를 환경변수에 두기

- 비밀번호를 코드에 그대로 적으면 GitHub에 올라가는 순간 공개된다. 그래서 앞 장의 **해시**를 쓴다: ① 내 비밀번호의 SHA-256 해시를 만들고 ② 그 값을 Vercel 환경변수 `PASSWORD_HASH`에 넣고 ③ 사이트에 비밀번호 입구를 단다
- 서버에는 비밀번호의 해시값만 두고, 입력값을 해시해 같은지 비교한다. 환경변수가 노출되어도 원문은 복원되지 않는다

💬 프롬프트

비밀번호 문자열을 SHA-256으로 해시해 16진수 문자열로 출력하는
아주 짧은 파이썬 스크립트를 만들어 실행해줘.
비밀번호는 실행할 때 내가 직접 입력할게 (코드나 대화에 남기지 않게).

복사

💬 프롬프트

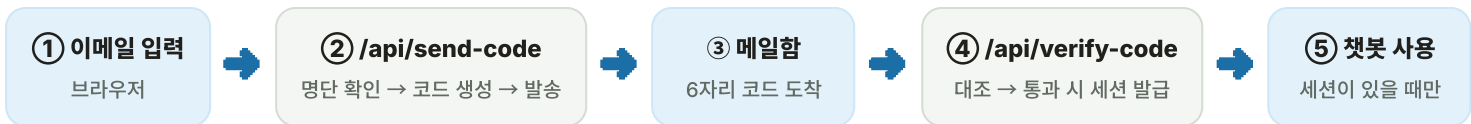
배포한 사이트에 비밀번호 입구를 달아줘.
1. 첫 화면에서 비밀번호 하나만 입력받는다.
2. 서버(/api/login)는 입력값을 SHA-256으로 해시해 환경변수
   PASSWORD_HASH와 비교하고, 일치하면 세션을 발급한다.
3. 세션이 없으면 화면과 모든 API가 동작하지 않아야 한다.
4. 틀렸을 때는 "비밀번호가 올바르지 않습니다"만 안내한다.
5. 원문 비밀번호는 어디에도 저장하거나 기록(log)하지 않는다.

복사

한계: 이것은 잠금이지 인증이 아니다. 비밀번호 하나로는 누가 들어왔는지 구분할 수 없다. 한 번 공유되면 통제가 끝나고, 사람별 접근 기록도 남지 않는다. 짧은 비밀번호의 단순 해시는 무차별 대입에 약하므로 문장형으로 길게 정한다. 실제 회원 비밀번호 저장에는 솔트와 느린 해시 함수(bcrypt, Argon2 등)를 쓴다. 혼자 또는 신뢰하는 소수가 쓰는 사이트까지가 적정 범위다.

인증과 권한

이메일 인증코드의 전체 흐름



서버만 아는 것 세 가지

- **허용 이메일 명단**: 명단에 없으면 코드 발송 자체를 하지 않는다
- **발송된 코드**: 서버가 잠시(예: 5분) 보관하고 대조 후 폐기한다
- **메일 발송 자격증명**: Gmail 앱 비밀번호. 환경변수에 둔다

이 방식이 본인 확인이 되는 이유

- 코드는 그 메일함으로만 간다. 메일함을 열 수 있는 사람을 그 이메일의 주인으로 보는 것이다. 즉 이메일 서비스(구글 등)의 로그인 보안에 의존한다
- 통과하면 서버가 **세션**(서명된 쿠키)을 발급한다. 이후 요청마다 이 쿠키가 함께 가므로 질문할 때마다 코드를 다시 받지 않는다

인증과 권한

발송 수단: Gmail SMTP + 앱 비밀번호

SMTP	메일 발송의 표준 프로토콜. 코드가 smtp.gmail.com:587에 접속해 "이 주소로 이 내용을 보내라"고 요청한다
앱 비밀번호	구글 계정 비밀번호 대신 프로그램 전용으로 발급하는 16자리 비밀번호. 계정 비밀번호를 코드나 서버에 두지 않기 위한 장치이고, 유출되면 그것만 폐기하고 재발급한다
발송 계정	**발송 전용 Gmail 계정을 새로 만든다** (예: nobullying.noreply@gmail.com). 유출되어도 잃는 것은 그 계정뿐이고, 개인 주소도 노출되지 않는다
발급	구글 계정 관리 → 보안 → 2단계 인증 켜기 → 검색창에 "앱 비밀번호" → 이름 입력 → 생성
한도	개인 Gmail 기준 하루 수백 통 수준. 인증코드 용도로는 충분하다

직접 메일 서버를 운영하지 않는 이유: 수신 측(Gmail, 네이버)은 평판이 없는 발신 서버의 메일을 스팸으로 분류하거나 거부한다. SPF·DKIM 설정을 갖추어도 신규 서버는 배달이 불안정하다. Gmail을 이용하면 구글 발신 서버의 평판으로 발송된다. 발송량이 늘면 Resend 같은 발송 서비스와 자기 도메인으로 옮긴다.

인증과 권한

구현 프롬프트와 환경변수

💬 프롬프트

이메일 인증코드 로그인을 추가해줘.

복사

구체적인 사항은 대화하면서 하나씩 추가한다

후속 지시 예

- ↩ 허용 명단은 환경변수 ALLOWED_EMAILS로
- ↩ 명단에 없으면 이유는 알려주지 마
- ↩ 코드는 5분만 유효, 3회 틀리면 폐기
- ↩ 인증 전에는 API도 동작하지 않게
- ↩ Gmail SMTP, 자격증명은 환경변수로
- ↩ 메일 제목은 "[판결문 챗봇] 인증코드 안내"

AI가 먼저 묻는 것(명단은 어디에 둘지, 메일은 무엇으로 보낼지)에 답하다 보면 같은 결과에 이른다. 다음 장의 고려 사항을 빠뜨리지 않았는지 마지막에 확인한다.

Vercel → 프로젝트 → Environment Variables

```
# .env 예시 (값은 각자의 것으로)
SMTP_USER=mybootcamp@gmail.com
SMTP_APP_PASSWORD=abcd efgh ijkl mnop
ALLOWED_EMAILS=kim@itm.kaist.ac.kr,park@itm.kaist.ac.kr
OPENROUTER_API_KEY=sk-or-v1-...
```

- 환경변수를 추가·변경한 뒤에는 **재배포**해야 반영된다
- “Sensitive”로 저장하면 대시보드에서도 값이 가려진다

인증과 권한

인증에서 고려할 사항: 각 요구가 막는 공격

고려 사항	하지 않으면 생기는 일	조치
API까지 막기	화면만 가리고 API를 열어 두면, 화면을 거치지 않고 API를 직접 호출하는 사람에게는 인증이 없는 것과 같다	모든 API에서 세션을 검사
명단 여부를 알려주지 않기	"등록되지 않은 이메일입니다"라고 답하면 공격자가 어떤 주소가 명단에 있는지 탐색할 수 있다(계정 열거)	성공·실패 응답을 같게
유효시간·시도 횟수 제한	6자리 코드는 100만 가지뿐이다. 제한이 없으면 무차별 대입으로 맞힐 수 있다	5분 유효, 3회 실패 시 폐기
발송 횟수 제한	같은 주소로 코드를 반복 요청해 상대 메일함을 채우거나 내 Gmail 발송 한도를 소진시킬 수 있다	주소별·IP별 분당 발송 제한
세션 보호	쿠키 서명 키가 약하거나 만료가 없으면 세션을 위조하거나 영구히 쓸 수 있다	서명 키를 환경변수에, 세션 만료 시간 설정, HttpOnly·Secure 쿠키
자격증명 위치	코드·GitHub에 앱 비밀번호가 남으면 내 이름으로 스팸이 발송된다	환경변수 + .gitignore 확인
명단도 개인정보	허용 명단의 이메일은 타인의 개인정보다. 화면·로그·저장소에 노출되면 유출이다	필요한 주소만, 화면 비노출, 목적 종료 시 삭제

Vercel의 서버리스 함수는 요청마다 새로 실행될 수 있어 메모리에 둔 코드가 사라질 수 있다. 코드 보관을 서명된 토큰이나 DB로 처리했는지 Claude Code에 확인한다. 보안 요구는 이렇게 하나씩 명시해야 반영된다.

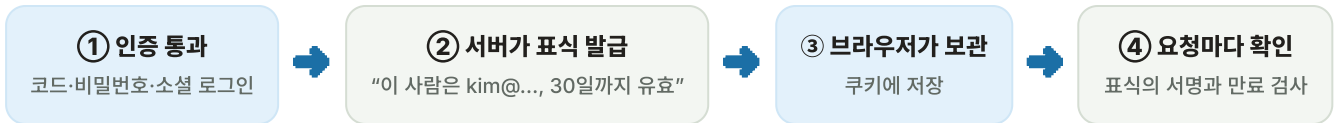
그 밖의 인증 방식: 무엇이 있고, 무엇을 맡기는가

방식	원리	장점	단점·주의	구현 난이도
공용 비밀번호 + 해시 (오늘)	비밀번호 하나를 아는 사람은 통과	가장 간단. 외부 서비스 불필요	사람을 구분하지 못함. 공유되면 통제 불가	하
이메일 인증코드 (오늘)	명단의 이메일로 6자리 코드를 보내고 확인	비밀번호를 저장하지 않음. 사람별 식별, 명단으로 관리	메일 발송 수단 필요, 메일 계정 보안에 의존	중하
매직 링크	코드 대신 일회용 로그인 링크를 메일로 보냄	입력할 것이 없음	메일 앱과 브라우저가 다르면 불편, 링크 전달 위험	중하
소셜 로그인 (OAuth 2.0 / OpenID Connect)	구글·카카오 등이 본인 확인을 하고 결과(신원 정보)를 내 서비스에 전달	비밀번호 관리를 대형 사업자에 맡김. 가입이 쉬움	제공자 앱 등록, 받는 개인정보 범위 결정 필요	중
사내 SSO (SAML, OIDC)	회사 계정 시스템(예: Microsoft Entra ID, Google Workspace)으로 로그인	입사·퇴사에 따라 접근이 자동으로 관리됨	회사 IT 부서의 등록·승인 필요	중
2단계 인증 (OTP 앱, SMS)	비밀번호에 더해 휴대폰의 일회용 숫자를 확인	비밀번호가 유출되어도 막을 수 있음	SMS는 번호 탈취(유심 복제)에 취약해 OTP 앱 권장	중
패스키 (FIDO2 / WebAuthn)	기기에 저장된 비밀 키로 서명. 지문·얼굴·PIN으로 잠금 해제	비밀번호가 없음. 가짜 사이트에서는 동작하지 않아 피싱에 강함	기기 분실 시 복구 절차 설계 필요	중상

오늘 gh 로그인에서 쓴 “브라우저에 일회용 코드를 입력하는” 방식(§1)도 OAuth의 한 형태(device flow)다. 비밀번호를 터미널에 입력하지 않는다. 소셜 로그인, SSO, 패스키는 직접 만들기보다 Supabase Auth, Firebase Auth, Auth0, Clerk 같은 인증 서비스를 연결하는 방식으로 쓴다.

인증과 권한

인증 다음: 통과한 사람을 어떻게 기억하나 (세션과 토큰)



- **서명된 쿠키 / 토큰(JWT 등)**: 서버가 표식의 내용에 비밀 키로 서명한다. 사용자가 내용을 "admin"으로 고치면 서명이 맞지 않아 거부된다. 서명 키는 환경변수에 둔다
- **만료**: 표식에는 유효기간을 둔다. 길면 편하지만 도난당한 표식이 오래 쓰인다. 업무용은 짧게(예: 하루~일주일)
- **쿠키 속성**: `HttpOnly`(자바스크립트가 읽지 못함), `Secure`(HTTPS로만 전송), `SameSite`(다른 사이트에서 보낸 요청에 붙지 않음)
- **로그아웃과 강제 만료**: 퇴사자·명단 삭제자의 기존 표식을 무효로 만드는 방법이 있어야 한다. 가장 단순한 방법은 서명 키를 바꾸는 것(모두 다시 로그인)
- **API 키와 세션의 차이**: API 키는 프로그램(서버)이 다른 서비스에 자신을 증명하는 값이고, 세션은 사람(브라우저)이 내 서비스에 자신을 증명하는 값이다. 둘 다 비밀이며 둘 다 유출되면 폐기한다

인증과 권한

권한(인가): 인증된 사람이 무엇을 할 수 있는가

1

역할 기반 접근 통제 (RBAC)

사람에게 역할(관리자, 일반 이용자, 열람자)을 주고, 역할마다 허용 기능을 정한다. 오늘의 챗봇: 이용자는 질문만, 관리자는 로그 열람까지(§7)

2

행 단위 보안 (RLS)

데이터베이스가 “이 사람은 자기 행만 볼 수 있다” 같은 규칙을 직접 집행한다. Supabase의 Row Level Security. 코드에 실수가 있어도 DB가 막는다

3

최소 권한 원칙

필요한 만큼만 준다. 서버가 쓰는 키도 마찬가지다: 로그를 쓰기만 하는 기능에 삭제 권한까지 있는 키를 쓰지 않는다. OpenRouter 키에는 사용 한도를 건다

오늘의 챗봇에 적용하면	이용자	관리자	서버 (환경변수의 키)
챗봇 질문	✅	✅	OpenRouter 호출 (한도 설정)
문답 로그 열람	❌	✅ 별도 비밀번호	Supabase 읽기
로그 삭제·허용 명단 변경	❌	대시보드에서 직접 (앱에는 기능을 두지 않음)	필요 없음

개인정보 보호법 관점: 「개인정보의 안전성 확보조치 기준」은 개인정보처리시스템에 대한 **접근 권한을 업무 수행에 필요한 최소한의 범위로 차등 부여**하고, 권한 변경 기록을 남기도록 정한다. 로그 열람을 관리자로 한정하는 설계가 이 요구에 해당한다.

SECTION 7

로깅: Supabase에 문답 기록 남기기

챗봇에 어떤 질문과 답변이 오가는지 클라우드 데이터베이스(Supabase)에 기록하고, 관리자만 여는 기록 화면을 만든다. 로그는 운영의 자산이자 개인정보 보호법상 책임의 대상이다.

1 Claude Code 시작하기
2 검색엔진 구현
3 RAG 적용
4 RAG의 한계
5 Vercel 배포
6 인증과 권한
7 로깅: Supabase
8 자동화 대상 고르기
9 정리와 과제

로깅: SUPABASE

로그의 세 측면: 자산 · 진단 · 책임

자산

사람들이 무엇을 자주 묻는가, 근거 판례가 빈약했던(데이터가 부족한) 주제는 무엇인가. 다음 개선 방향이 기록에서 나온다

진단

"어제 이상한 답이 나왔다"는 문의에 답하려면 그때의 질문과 답변이 남아 있어야 한다. 호출 횟수로 비용도 검증한다

책임

질문에는 이용자의 관심사, 때로는 민감한 사정이 담긴다. 법률 질문이면 더욱 그렇다. 필요한 것만 수집하고, 수집 사실을 알리고, 목적이 끝나면 지운다

저장소 후보	평가	결론
Vercel Logs	이미 있지만 며칠 뒤 사라지고 검색·집계가 어렵다	오류 확인용
서버에 파일로	서버리스 함수에는 고정 디스크가 없다. 실행이 끝나면 파일도 사라진다	불가
클라우드 데이터베이스	영구 저장, 표로 조회, 어디서나 접속. 서비스 로그의 표준적 위치	**채택 (Supabase)**

로깅: SUPABASE

Supabase 준비: 프로젝트, 키, 표 하나

- **가입·프로젝트 생성**: supabase.com → GitHub 계정으로 가입 → New project. 이름은 자유, Database password는 생성 후 보관, **Region은 가까운 곳**(아시아 태평양, 서울 리전이 있으면 서울), Security는 기본값
- **접속 정보 두 가지**: 프로젝트 홈의 Project URL, 그리고 API Keys의 키
- **Publishable key가 아니라 Secret key**: Publishable key(`sb_publishable_…`)는 브라우저에 노출해도 되는 키로, 행 단위 보안 규칙(RLS)과 함께 쓴다. 우리 구조는 서버리스 함수만 DB에 접근하므로 서버 전용 **Secret key**(`sb_secret_…`)를 쓴다. 환경변수로만 다룬다
- **Vercel 환경변수 3개**: `SUPABASE_URL`, `SUPABASE_KEY`, `ADMIN_PASSWORD_HASH`(사이트 접속용 해시와 다른 값)

SQL Editor → New query → Run

```
create table if not exists public.chat_logs (
  id bigint generated always as identity primary key,
  created_at timestamptz not null default now(),
  question text not null,
  answer text not null,
  cost_krw numeric(12,4),
  duration_sec numeric(8,2)
);
create index if not exists chat_logs_created_at_idx
  on public.chat_logs (created_at desc);
alter table public.chat_logs enable row level security;
```

"Success. No rows returned"는 빈 표가 만들어졌다는 뜻이다. 칸: 시각·질문·답변·비용·소요 시간. **이용자 신원(이메일, IP)은 저장하지 않는다.**

로깅: SUPABASE

한 줄 지시로 시작해 대화로 정한다: 문답 로그와 관리자 화면

💬 프롬프트

챗봇에 문답 로그 기능을 추가해줘.

복사

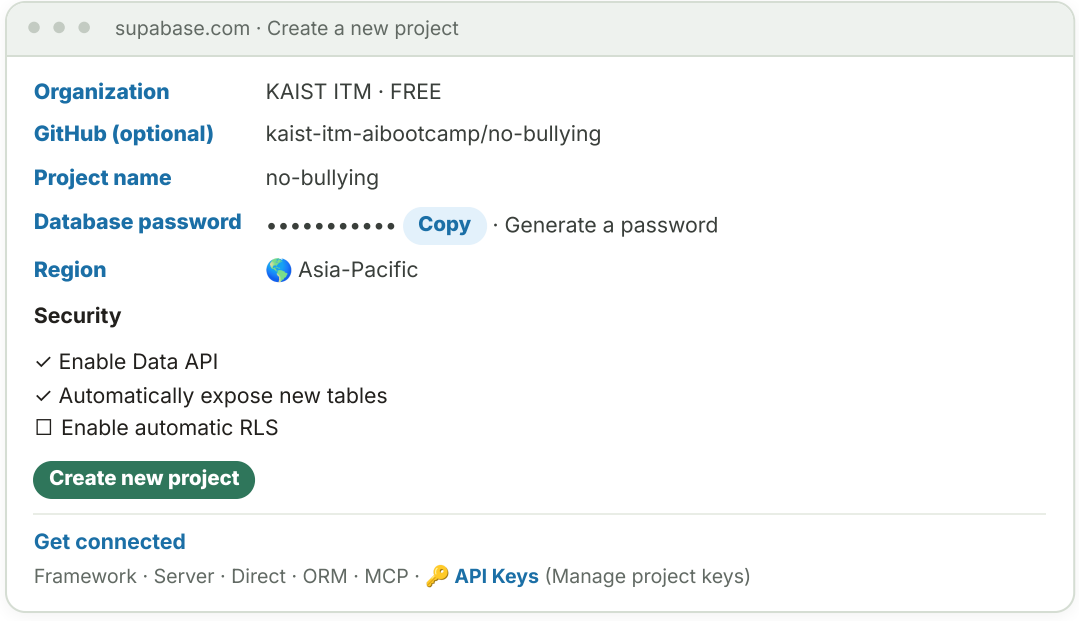
이후 AI와 대화하며 정한 것

AI의 질문	내 답
무엇을 기록할까요?	시각, 질문, 답변, 비용(원화 환산), 소요 시간. 이용자 신원은 기록하지 않음
어디에 저장할까요? (서버리스 함수에는 파일이 남지 않음)	Supabase에 저장
기록은 어디서 보시겠어요?	화면 우측 상단 톱니바퀴 → 관리자 화면, 최근 순 표
관리자 화면은 어떻게 보호할까요?	입력한 비밀번호를 해시해 환경변수 ADMIN_PASSWORD_HASH와 비교

- **관문을 둘로**: 접속 비밀번호(또는 이메일 인증)는 이용자와 공유하지만, 기록은 관리자만 본다.
- **비용은 실제 청구액**: OpenRouter가 응답에 그 요청의 실제 달러 비용을 알려준다. 질문 한 번에 최대 3회 호출(임베딩 + 이어지는 대화의 검색문 재작성 + 답변)을 더해 저장. 실측 질문당 1.2~1.5원
- **기록 실패는 조용히 건너뛴다**: 로그 때문에 본 기능이 멈추면 안 되기 때문이다. 반대로 기록이 안 쌓여도 겉으로 드러나지 않는다. 설정을 바꾼 뒤에는 질문 → Table Editor 확인을 한 번 거치고, 안 되면 `/api/health`로 무엇이 빠졌는지 확인한다
- 표 위에 건수·비용 합계·평균 소요 시간을 요약하면 열자마자 운영 현황이 보인다

로깅: SUPABASE

실제 화면: Supabase 프로젝트 만들기, 그리고 톱니바퀴 뒤의 기록 표





no-bullying-gamma.vercel.app · ⚙ 관리자

건수 12 | 비용 합계 16.3원 | 질문당 1.36원 | 평균 18.4초

시각	질문	답변	비용	시간
10/14 21:05	회사에도 책임을 물을 수 있나요?	네, 회사(사용자)에게도...	1.2원	21.3초
10/14 20:58	녹음 파일이 몇 개나 필요한가요	구체적인 숫자 기준은...	1.5원	17.9초

화면 구성을 보여주기 위한 예시 행

- 일반 이용자에게 보이는 것은 톱니바퀴 하나뿐이다. 비밀번호를 모르면 기록에 닿을 수 없다
- 같은 데이터를 Supabase 대시보드의 Table Editor에서도 볼 수 있다. 관리 화면에 문제가 생겨도 데이터는 DB에 남아 있다

로깅: SUPABASE

로그와 개인정보 보호법: 6강의 원칙을 적용하면

쟁점	질문 로그에서의 의미	설계 조치
개인정보 해당 여부	신원을 저장하지 않아도 질문 본문에 이름·사건 정황이 들어갈 수 있다. 다른 정보와 결합해 식별 가능하면 개인정보다	신원 칸을 두지 않음. 질문 본문은 개인정보가 섞일 수 있다는 전제로 다룬다
민감정보 (제23조)	괴롭힘으로 인한 우울증 같은 건강 정보가 질문에 담길 수 있다	입력 화면에 "실명·건강정보를 입력하지 마십시오" 안내
수집 고지와 처리방침 (제15조·제30조)	무엇을 왜 얼마나 보관하는지 알려야 한다	"문답이 서비스 개선을 위해 기록됩니다" 고지, 처리방침 게시
보관 기간과 파기 (제21조)	목적이 끝난 기록을 계속 두면 위반이 될 수 있다	예: 90일 지난 행 자동 삭제 지시
안전조치 (제29조)	기록 열람 권한을 통제해야 한다	관리자 비밀번호 해시, Secret key는 환경변수, 브라우저의 직접 DB 접근 차단 (RLS)
처리위탁·국외 이전 (제26조·제28조의8)	Supabase, Vercel, OpenRouter와 그 뒤의 모델 제공자가 모두 처리에 관여한다	리전 선택, 위탁 사실과 이전 국가 공개, 데이터 보존하지 않는 제공자 선택

개인 실습 수준에서는 과한 장치처럼 보이지만, 업무 서비스로 옮기는 순간 모두 필요한 항목이다

SECTION 8

내 업무에서 AI 자동화 대상 고르기

직무 전체가 아니라 과업 단위로 업무를 나누고, 가치와 위험을 기준으로 첫 번째 대상을 고른다.
그다음 스킬로 만들지, 웹앱으로 만들지 정한다.

1 Claude Code 시작하기
2 검색엔진 구현
3 RAG 적용
4 RAG의 한계
5 Vercel 배포
6 인증과 권한
7 로깅: Supabase
8 자동화 대상 고르기
9 정리와 과제

자동화 대상 고르기

출발점: 직무가 아니라 과업(task) 단위로 나눈다

- “계약 검토를 AI로 자동화한다”는 너무 크다. 계약 검토는 여러 과업의 묶음이고, 과업마다 AI가 잘하는 정도와 틀렸을 때의 비용이 다르다
- 과업은 **입력 → 처리 → 출력**이 분명한 한 덩어리의 일이다. 출력이 누구에게 가는지(나, 동료, 고객, 기관)까지 적는다

과업 (예: 사내 법무팀의 계약 검토)	입력	출력과 받는 사람	AI 적합도
검토 요청 접수·분류 (계약 유형, 긴급도)	요청 메일, 첨부 계약서	분류표 → 나	높음
표준 계약서와 차이점 비교	상대방 초안, 사내 표준안	변경 조항 목록 → 나	높음
사내 체크리스트 대조 (손해배상 한도, 준거법 등)	계약서, 체크리스트	위험 조항 표시 → 나	높음 (기준이 문서화된 경우)
수정 문구 초안 작성	위험 조항, 과거 협상 사례	수정안 → 상대방	중간 (사람 검토 필수)
현업 부서 질문 응대 (“이 조항 괜찮나요?”)	질문, 사내 규정	답변 → 동료	중간 (RAG 후보)
최종 승인 판단·서명	전체 검토 결과	결재 → 회사	사람 (책임 주체)

여러분의 업무도 같은 표로 여러 개의 과업으로 나눌 수 있다. AI에게 시켜 볼 것.

자동화 대상 고르기

후보 과업을 평가하는 6가지 기준

1

빈도와 소요 시간

주 1회 이상, 회당 30분 이상이면 절감 효과가 분명하다. 1년에 한 번 하는 일은 자동화보다 그때 Chat에 묻는 편이 낫다

2

입력이 디지털 텍스트인가

메일, 문서, 표, PDF면 바로 시작할 수 있다. 종이·구두·사내 시스템 화면에만 있는 정보는 수집 단계부터 일이 된다

3

기준이 문서화되어 있는가

체크리스트, 규정, 과거 사례가 있으면 AI에게 줄 수 있다. 판단 기준이 머릿속에만 있다면 그것을 적는 일이 첫 작업이다(3강의 스킬)

4

결과를 검증하기 쉬운가

결과가 맞는지 확인하는 데 드는 시간이 직접 하는 시간보다 훨씬 짧아야 한다. 검증이 어려운 일은 AI가 틀려도 알아채기 어렵다

5

틀렸을 때의 비용

내부 초안이 틀리면 고치면 된다. 고객·기관에 나가는 문서, 금전·권리에 영향을 주는 판단이 틀리면 되돌리기 어렵다

6

데이터의 민감도

공개 자료 / 사내 일반 / 영업비밀·고객 개인정보·민감정보. 민감할수록 쓸 수 있는 도구와 배포 방식이 줄어든다

1~4는 가치(얻는 것), 5~6은 위험(잃을 수 있는 것)이다

자동화 대상 고르기

가치 × 위험으로 첫 번째 대상을 고른다



- 처음부터 가장 큰 일을 고르지 않는다. **작게 성공하고, 검증 방법을 익힌 뒤 범위를 넓힌다**. 과제의 웹앱이 “간단해도 무방”한 이유다

자동화 대상 고르기

어떤 형태로 만들 것인가: Chat · 스킬 · 웹앱

	Chat에 그때그때	스킬 (SKILL.md)	웹앱 (Vercel)
누가 쓰나	나	나 (또는 같은 도구를 쓰는 동료)	링크를 받은 다른 사람, 여러 명
어울리는 일	드물고 매번 다른 일	내가 반복하는 절차. 내 PC의 파일을 읽고 결과 파일을 만드는 일	입력 폼 → 결과가 정해진 일. 검색, RAG Q&A, 계산·양식 생성
담는 것	프롬프트	절차, 판단 기준, 출력 양식, 예시	화면, 서버 함수, 데이터, 인증, 로그
AI 호출	내 구독 계정	내 구독 계정 (Cowork, Claude Code)	API 키, 사용량만큼 과금
추가로 생기는 책임	입력 데이터 관리	입력 데이터 관리	+ 비밀 관리, 접근 통제, 이용자 데이터, 비용 통제

- **스킬**은 3강 §9에서 만들었다. 반복 절차를 SKILL.md 한 장에 적고, description에 "언제 이 스킬을 쓰는지"를 분명히 적는다
- **웹앱**은 오늘 만든 구조를 재사용한다. 정적 페이지로 충분하면 GitHub Pages로도 된다. API 키가 필요하면 Vercel

> 같은 과업도 형태에 따라 법적 검토 범위가 달라진다. 나만 쓰는 스킬은 "내가 이 데이터를 이 AI 도구에 넣어도 되는가"가 주된 질문이고, 남이 쓰는 웹앱은 "이용자의 데이터를 내가 처리해도 되는가"가 추가된다.

자동화 대상 고르기

직군별 예시: 업무 자체는 간단해도 된다

직군	스킬 예시	웹앱 예시	먼저 확인할 점
법무	상대방 계약서를 사내 표준계약서와 대조해 위험 조항 표를 만드는 스킬	사내 규정·표준계약 RAG Q&A (현업용)	계약서의 영업비밀
인사	직무기술서 → 채용공고 초안 스킬	휴가·복리후생 규정 Q&A 챗봇	지원자 개인정보, 채용 판단에 쓰지 않을 것
엔지니어·R&D	회의록 → 결정 사항·할 일·담당자 표	사내 기술문서·특허 유사 검색기	미공개 기술정보, 고객사 자료
공공	정해진 양식에 따른 문서 초안 작성	법령·지침 조문 검색기	비공개 정보 입력 금지, 공공부문 가이드라인
영업·기획	주간 실적 엑셀 → 보고서 요약	견적 계산기, 제안서 근거 자료 검색	고객 정보, 가격 정보
연구·교육	논문 PDF → 정해진 양식의 리뷰 노트	강의 자료 Q&A, 실험 장비 예약 안내	저작물 이용 범위, 학생 정보
개인	카드 명세서 → 가계부 분류	가족 여행 일정 공유 페이지, 운동 기록장	가족 개인정보, 공개 범위

자동화 대상 고르기

지금 해 보기: AI와 함께 후보를 찾고 한 줄로 정의한다 (5분)

💬 프롬프트

복사

내 직무는 [직무]야. 이 직무를 맡은 사람이 주로 하는 일을 먼저 리스트업해줘.
빠진 것이나 내 상황과 다른 것은 내가 고쳐 줄게.
확정된 일들을 입력 → 처리 → 출력이 분명한 과업 단위로 나누고, 과업마다
빈도, 회당 소요 시간, 입력 형태, 판단 기준의 문서화 정도, 결과 검증의 쉬움,
틀렸을 때의 비용, 데이터 민감도를 표로 평가해줘.
그다음 "지금 시작" 후보 3개를 고르고, 각각 스킬과 웹앱 중 어느 형태가
맞는지 이유와 함께 제안해줘. 회사 기밀이나 개인정보가 필요한 과업은
표시만 하고 구체적인 내용은 묻지 마.

한 줄 정의 양식

"나는 **[누가]**가 **[입력]**을 넣으면 **[출력]**을 내주는 **[스킬/웹앱]**을 만든다. 이것으로 **[과업]**에 드는 시간을 **[얼마]** 줄인다."

이어서 스스로 묻기

① 이 과업에 들어가는 데이터를 이 AI 도구에 넣어도 되는가 ② 결과가 틀리면 누가 어떤 피해를 입는가 ③ 결과를 내가 어떻게 검증하는가. 답이 애매한 지점이 과제 발표의 토론 주제다.

이 프롬프트 자체에 업무 정보가 들어간다. 회사가 허용한 AI 도구를 쓰거나, 과업 이름 수준으로만 적는다.

SECTION 9

정리와 과제

실습 내용을 한 장으로 정리하고, 10/29(9강) 수강생 발표 (2)의 과제를 확인한다.

1 Claude Code 시작하기
2 검색엔진 구현
3 RAG 적용
4 RAG의 한계
5 Vercel 배포
6 인증과 권한
7 로깅: Supabase
8 자동화 대상 고르기
9 정리와 과제

정리와 과제

실습 정리

1 Claude Code
git·gh 설치, 폴더를 열고 지시 한 번으로 저장소 생성. 비밀 파일은 올리기 전에 확인. 승인 수준은 다루는 자료로 정한다

2 임베딩 검색
키워드는 동의어·표현·의도에서 실패한다. 임베딩은 의미를 찾는다. 질의 임베딩은 색인과 같은 모델로

3 백엔드
새 질의의 실시간 임베딩과 키 은닉 때문에 필요하다. 정적 페이지에는 비밀을 둘 곳이 없다

4 계획서부터 대화로
원하는 것만 서술 → AI 계획서 → 피드백 → 구현. 확인 지점과 "하지 않는 것"을 명시.
PLAN·DECISIONS·README·CLAUDE.md

5 RAG
검색 → 질문 앞에 근거 문서 추가 → 근거만으로 답변·인용. 인용 번호는 프로그램이 검증. 답이 이상하면 검색부터 확인

6 RAG의 한계
상용 법률 도구도 환각 17~33%. 인용이 붙어도 뒷받침하지 않을 수 있다. 검색 실패와 생성 실패를 나눠 디버깅하고, 최종 확인은 원문으로

7 Long Context · 하이브리드
소규모·혼자 하는 분석은 통째로, 서비스·대규모는 RAG, 요지 + 전문 골라 읽기는 하이브리드

8 Vercel 배포
서버리스 = 서버를 내가 관리하지 않음. push가 곧 배포. 비밀은 서버의 환경변수에

9 인증과 권한
해시는 일방향·고정 길이. 혼자 쓰면 비밀번호 해시 잠금, 명단 관리는 이메일 인증코드(API까지 막기, 시도·발송 제한). 그 밖에 SSO·소셜 로그인·패스키, 권한은 최소한으로

10 로깅
Supabase에 시각·질문·답변·비용·시간. 신원은 저장하지 않음. 관리자 화면은 별도 비밀번호. 로그는 자산이자 책임

11 데이터의 경로
브라우저 → Vercel → OpenRouter → 모델 제공자 → Supabase. 각 단계가 처리위탁·국외 이전 판단의 대상

정리와 과제

과제: 나의 업무에 AI를 도입한다 (10/29 수강생 발표 2)

(i)

스킬로 자동화한 사례 1건

- 업무 또는 사적 작업 어느 쪽이든 된다
- SKILL.md 한 장(절차, 판단 기준, 출력 양식)과 실제로 돌린 결과물을 제출한다
- 스킬을 쓰기 전과 후의 소요 시간, 결과물의 차이를 비교한다

(ii)

Vercel로 배포한 웹앱 1건

- 업무 자체는 간단해도 된다. 기능 하나를 끝까지 동작하게 하는 것이 목표다
- 오늘의 검색엔진·RAG를 내 데이터로 바꾸어도 되고, 계산기·양식 생성기처럼 전혀 다른 앱이어도 된다
- 배포 URL 1개 제출. 다른 사람의 기기에서 열리는지 확인한다. 필요하면 인증(§6)과 로그(§7)를 붙인다

핵심 토론 주제: "이것을 해도 되는가?" 두 사례 각각에 대해 ① 법적·규정상 애매한 점이 있었는지(데이터를 AI 도구에 넣어도 되는지, 공개해도 되는지, 결과를 그대로 써도 되는지) ② 어떻게 판단했고 어떻게 극복했는지 ③ 그래도 남은 위험은 무엇인지 발표한다. 애매한 점이 없었다면 왜 없다고 판단했는지를 설명한다.

발표는 한 사람당 5분 안팎. 결과물보다 판단 과정을 중심으로 한다.

정리와 과제

웹앱 아이디어: 무엇이든, 단 내 것으로

방향	예시	재사용할 내용
검색엔진	내 분야 문서(특허·논문·리뷰·사내 규정) 유사 검색기. 3강 과제의 특허 임베딩이 있다면 바로 재료가 된다	§2
RAG 질의응답	내 자료를 근거로 답하는 챗봇 (사내 규정 안내, 논문 Q&A)	§3, §5
RAG 방식 문서 작성기	"작성 요청"을 받으면 관련 자료를 검색해 근거를 모아 초안을 써 주는 도구. 질문에 답하는 대신 문서를 생성	§3, §5, 3강 스킬
완전히 다른 것	계산기, 시뮬레이터, 양식 생성기, 일정 공유 페이지. 3강의 정적 웹앱에 서버 기능을 얹은 무엇이든	3강 + §5

💬 프롬프트

```
이런 웹앱을 만들려고 해: [나의 한 줄 정의].
바로 코드를 쓰지 말고, 먼저 계획서를 작성해줘 - 화면 구성, 필요한
API(백엔드 함수), 데이터·파일 구조, 만들 순서. 내가 검토한 뒤 시작하자.
궁금한 점이 있으면 먼저 질문해줘.
```

복사

- **범위는 작게**: 동작하는 작은 것이 거창한 미완성보다 낫다. 기능 하나를 정해 그것만 끝까지
- 로컬에서 먼저 돌려 보고, 키는 처음부터 환경변수로
- 문서 네 파일(PLAN·DECISIONS·README·CLAUDE.md)을 남긴다. 합격선: 처음 보는 사람이 README만 보고 실행할 수 있는가
- 결승선: 배포된 URL을 다른 기기(휴대폰)에서 열어 보는 순간

막히면 막힌 화면을 AI에게 보여주고 "여기서 왜 안 되는 거야?"라고 묻는다. 범위를 줄이는 것도 좋은 선택이다.

정리와 과제

“해도 되나” 애매한 점의 예시와 극복 수단

애매한 점	극복 수단의 예
회사 내부 문서를 외부 AI 서비스에 넣어도 되는가	유사한 형태의 문서를 가상 데이터로 만들어서 사용
고객·동료의 개인정보가 들어 있다	가명처리 또는 가상 데이터로 개발하고 실제 데이터는 승인 후에, 필요한 항목만 사용, 처리위탁 여부 검토
웹앱을 공개 URL에 올려도 되는가	접근 통제 수준 선택(인증, 비밀번호 잠금), 공개 가능한 데이터만 탑재, 사내망·사내 계정 연동
AI가 만든 결과를 그대로 외부에 보내도 되는가	사람 검토 단계를 절차에 넣기, 출처 인용 강제, AI 생성 사실의 고지(인공지능 기본법의 투명성 의무 검토)
남의 자료(판례 DB, 유료 보고서, 웹페이지)를 넣어도 되는가	이용 약관과 저작권 검토, 공공 데이터·자체 자료로 대체, 내부 분석 목적으로 한정
법률·의료 등 자격이 필요한 판단을 하는 것처럼 보인다	정보 제공으로 범위 한정, 전문가 연결, 고지 문구 고정 표시
API 비용·남용	키 사용 한도, 인증, 발송·호출 횟수 제한, 로그로 사용량 확인

정리와 과제

제출물과 발표 구성

제출물 (발표 전까지 KLMS)

1. 스킬: SKILL.md 내용을 소개하고 실행 예시를 보여주는 HTML 페이지 (1페이지)
2. 웹앱: Vercel URL 1개

> **배포 전 점검**: ① 데이터에 개인정보·영업비밀이 없는가 ② 공개해도 되는 데이터인가 ③ 비용 한도를 걸었는가

발표 구성 (5분)

1. 내 업무와 고른 과업
2. 왜 이 형태(스킬/웹앱)를 골랐나
3. 시연: 입력 → 출력
4. "해도 되나": 애매한 점, 판단 근거, 극복 방법
5. 신뢰성: 결과가 맞는지 어떻게 확인했나, 틀린 사례가 있었나
6. 남은 위험과 실제 업무에 쓰려면 더 필요한 것